



## HOJA DE PROBLEMAS 2: UNIDAD ARITMÉTICO-LÓGICA (ALU)

1. Se desea diseñar una unidad aritmético-lógica de 4 bits que realice las siguientes operaciones, reguladas por la señal de selección  $OP$  (2 bits):

| $OP_1$ | $OP_0$ | Operación                            |
|--------|--------|--------------------------------------|
| 0      | 0      | $X + Y$ (suma aritmética)            |
| 0      | 1      | $X - Y$ (resta aritmética)           |
| 1      | 0      | $X \text{ AND } Y$ (producto lógico) |
| 1      | 1      | $X \text{ OR } Y$ (suma lógica)      |

### Solución:

Es una ALU parecida a la vista en clase. La construiremos con 4 miniALUs de 1 bit para seguir con el diseño jerárquico y modular. Cada una de ellas calculará 1 bit del resultado, de tal forma que hablaremos de miniALU de peso  $i$  ( $0 \leq i \leq 3$ ). Para elegir el resultado final, usaremos un multiplexor de 4 a 1, con dos entradas de control, que serán precisamente  $OP_1$  y  $OP_0$ , en los pesos 1 y 0, respectivamente.

Para sumar y restar usaremos el mismo sumador completo de 1 bit, donde el bit  $Y_i$  puede ir afirmado (suma) o negado (resta). En el caso de la resta, debemos poner un acarreo de entrada  $C_{in_0} = 1$  para el bit 0 (los otros provienen de la miniALU anterior por la propagación habitual del acarreo).

Por otra parte, como el sumador completo sirve para 2 operaciones que necesitan un valor diferente para el segundo operando, añadiremos un multiplexor 2 a 1 para seleccionar el adecuado. Colocaremos  $Y_i$  en la entrada de peso 0, mientras que pondremos  $\bar{Y}_i$  en la de peso 1. La siguiente tabla indica qué valor se necesita en cada caso:

| $OP_1$ | $OP_0$ | Operación en el sumador | Valor necesario en segundo operando | Señal de selección del multiplexor de $Y_i$ |
|--------|--------|-------------------------|-------------------------------------|---------------------------------------------|
| 0      | 0      | $X_i + Y_i + 0$         | $Y_i$                               | 0                                           |
| 0      | 1      | $X_i + \bar{Y}_i + 1$   | $\bar{Y}_i$                         | 1                                           |
| 1      | 0      | —                       | —                                   | X                                           |
| 1      | 1      | —                       | —                                   | X                                           |

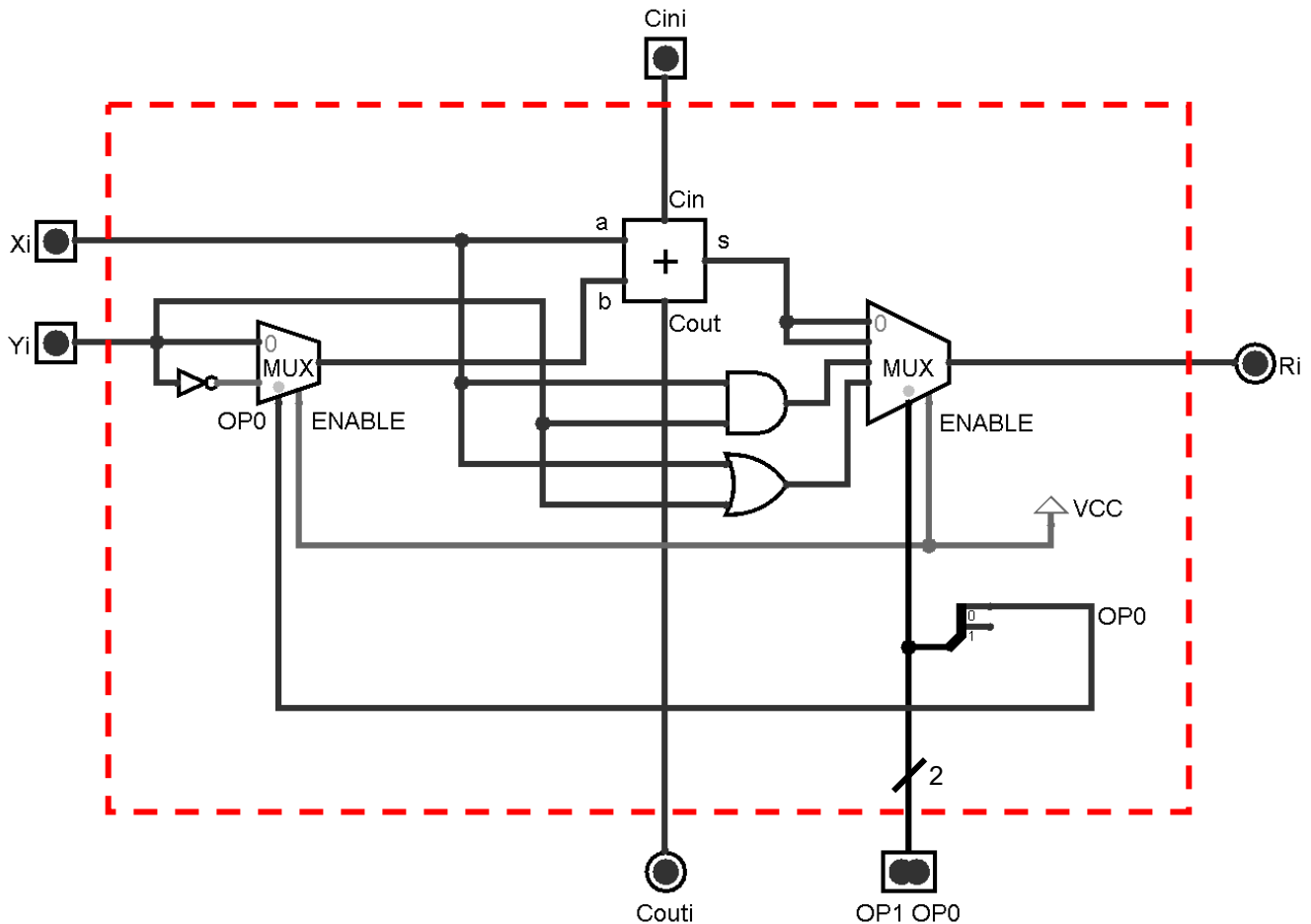
| $OP_1 \backslash OP_0$ | 0 | 1 |
|------------------------|---|---|
| 0                      | 0 | 1 |
| 1                      | X | X |

Puede verse por Karnaugh que la señal de selección de este multiplexor vale lo mismo que  $OP_0$ .

Sin embargo, en las puertas AND y OR que hacen las dos últimas operaciones, no necesitamos usar ningún multiplexor para seleccionar sus operandos, ya que, en este caso, siempre serán  $X_i$  e  $Y_i$ , respectivamente. Tampoco se necesita un multiplexor específico para  $X_i$ , ya que siempre se usa el mismo valor como entrada del sumador completo tanto en la suma como en la resta.

Cada miniALU proporciona un acarreo de salida  $C_{out_i}$  que servirá como acarreo de entrada  $C_{in_{i+1}}$  para la siguiente miniALU. La única excepción es el acarreo  $C_{out_3}$ , que debe ignorarse y no forma parte del resultado debido a que los datos están representados en complemento a 2. Y en las operaciones lógicas tampoco tiene sentido.

En resumen, el diseño de la miniALU de peso  $i$  ( $0 \leq i \leq 3$ ) sería el siguiente:



Este dibujo se ha realizado con el programa gratuito Logisim ([http://www.cburch.com/logisim/index\\_es.html](http://www.cburch.com/logisim/index_es.html)) y se ha editado posteriormente a mano para mejorar la claridad del circuito. Las entradas vienen indicadas con cuadrados, mientras que las salidas se han pintado con círculos. El recuadro rojo muestra las fronteras de la miniALU de 1 bit, con lo cual se hacen evidentes sus entradas y sus salidas (a modo de caja negra).

Se ha usado este programa solo con fines pedagógicos. No se requiere en ningún momento en esta asignatura aprender a manejarlo.

Se muestra a continuación el funcionamiento de esta miniALU de 1 bit mediante un pseudolenguaje de descripción de hardware, que servirá para aclarar el diagrama del circuito:

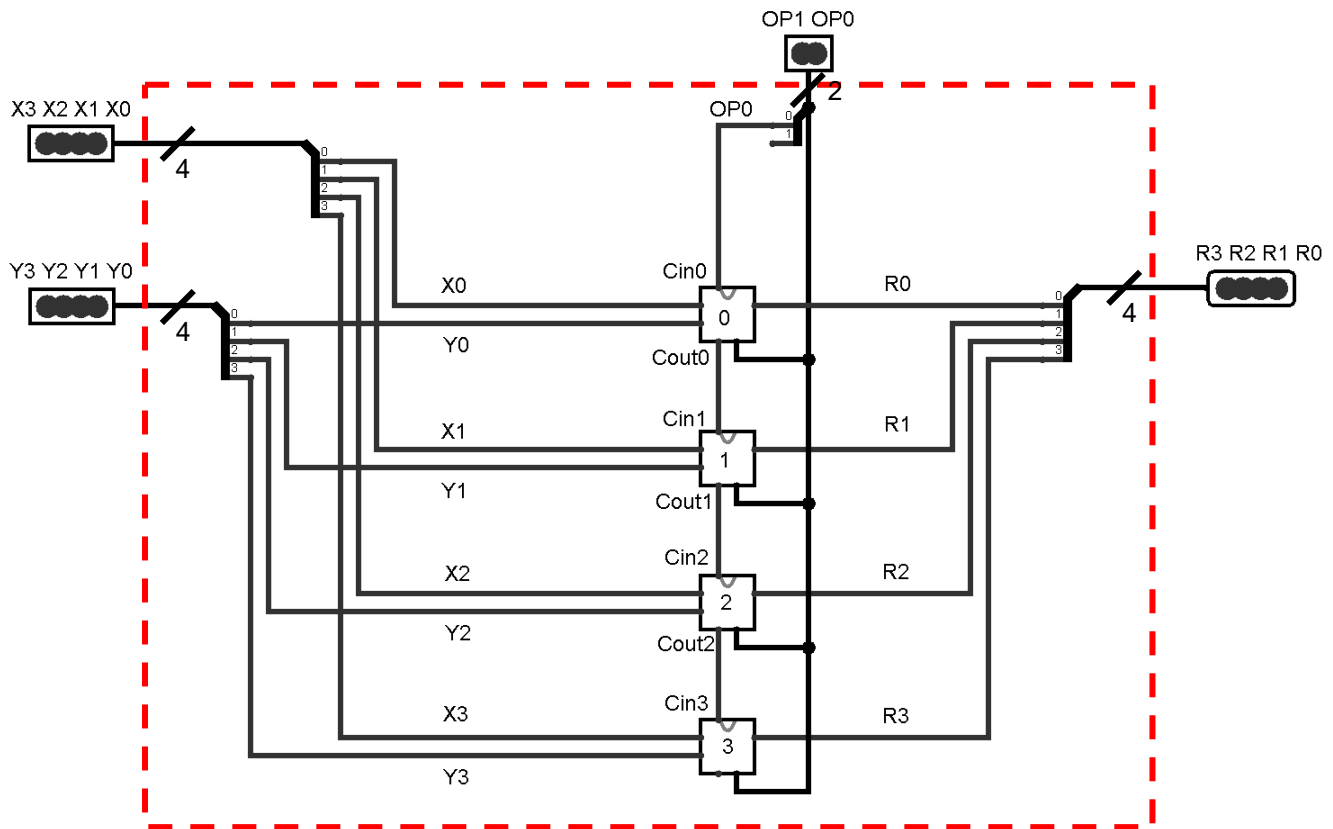
```
(bit Couti, bit ri) ALU_1_bit(bit xi, bit yi, bit cini, bit op[2])
{
    bit enable = 1
    bit entradasMux2a1_B[2]
    entradasMux2a1_B[0] = yi
    entradasMux2a1_B[1] = NOT yi
    bit b = mux2a1(entradasMux2a1_B, op[0], enable)
    bit Couti, s, ri
    (Couti, s) = sumadorCompleto_1_bit(xi, b, cini)
    bit entradasMux4a1_R[4]
    entradasMux4a1_R[0] = s
    entradasMux4a1_R[1] = s
    entradasMux4a1_R[2] = xi AND yi
    entradasMux4a1_R[3] = xi OR yi
    ri = mux4a1(entradasMux4a1_R, op, enable)
    return (Couti, ri)
}
```

En cuanto al acarreo de entrada inicial, tiene la siguiente tabla de verdad:

| $OP_1$ | $OP_0$ | $C_{in0}$ |
|--------|--------|-----------|
| 0      | 0      | 0         |
| 0      | 1      | 1         |
| 1      | 0      | X         |
| 1      | 1      | X         |

Esta es la misma tabla de verdad que la del multiplexor del bit  $B_i$  en cada miniALU, con lo que se deduce que  $C_{in0}$  también vale  $OP_0$ .

Por tanto, la ALU completa sería la siguiente:



En este caso, puede verse que se ha replicado 4 veces la miniALU de 1 bit, de tal forma que cada una de ellas calcula 1 bit del resultado final. El recuadro rojo indica las fronteras de la ALU completa.

Pseudolenguaje de descripción de hardware de la ALU completa:

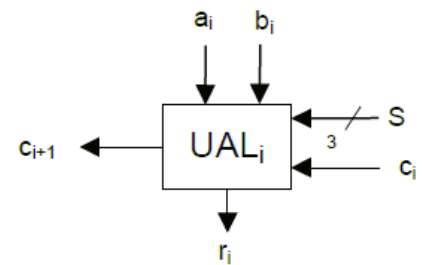
```
(bit r[4]) ALU_4_bits(bit x[4], bit y[4], bit op[2])
{
    bit r[4]
    bit cout[4]
    (cout[0], r[0]) = ALU_1_bit(x[0], y[0], op[0], op)
    for (int i = 1; i < 4; i++)
    {
        (cout[i], r[i]) = ALU_1_bit(x[i], y[i], cout[i-1], op)
    }
    return (r)
}
```

2. Se desea diseñar una unidad aritmético-lógica para datos de 8 bits que realice las siguientes operaciones, reguladas por la señal de selección  $S$  (3 bits):

| $S_2$ | $S_1$ | $S_0$ | Operación                                  |
|-------|-------|-------|--------------------------------------------|
| 0     | 0     | 0     | $A \text{ OR } B$ (suma lógica)            |
| 0     | 0     | 1     | $A \text{ AND } B$ (producto lógico)       |
| 0     | 1     | 0     | $A \text{ XOR } B$ (suma lógica exclusiva) |
| 0     | 1     | 1     | $A + B$ (suma aritmética)                  |
| 1     | 0     | 1     | $A \text{ NOR } B$ (suma lógica negada)    |
| 1     | 1     | 1     | $A - B$ (resta aritmética)                 |

- a) Utilizando multiplexores, sumadores binarios elementales y/o puertas lógicas, materializar una celda elemental de la UAL (celda que llamaremos  $UAL_i$ ) para datos de entrada de 1 bit. La celda  $i$  tendrá las siguientes entradas y salidas:

- Operandos de entrada (1 bit cada uno):  $a_i$  (primer operando) y  $b_i$  (segundo operando).
- Entrada de selección de operación:  $S = (S_2, S_1, S_0)$ .
- Acarreo de entrada:  $c_i$ .
- Resultado de salida:  $r_i$ .
- Acarreo de salida:  $c_{i+1}$ .



- b) Utilizando celdas  $UAL_i$ , materializar una UAL para datos de 8 bits que, además del resultado  $R$ , genere bits indicadores de resultado nulo ( $Z$ ), negativo ( $N$ ) y desbordamiento para complemento a 2 ( $V$ ).

- c) En la UAL de 8 bits, añadir la circuitería necesaria para generar las siguientes señales:

- $EQ$ : es una señal que vale 1 cuando los dos operandos de entrada son iguales, y 0 en caso contrario.
- $NE$ : es una señal que vale 1 cuando los dos operandos de entrada son distintos, y 0 en caso contrario.
- $LT$ : es una señal que vale 1 cuando el primer operando es menor que el segundo, y 0 en caso contrario.
- $LE$ : es una señal que vale 1 cuando el primer operando es menor o igual que el segundo, y 0 en caso contrario.

- $GT$ : es una señal que vale 1 cuando el primer operando es mayor que el segundo, y 0 en caso contrario.
- $GE$ : es una señal que vale 1 cuando el primer operando es mayor o igual que el segundo, y 0 en caso contrario.

### Solución:

a) Como es habitual en este tipo de ejercicios, realizaremos un diseño jerárquico comenzando por la miniALU para datos de 1 bit. En este caso hay 6 operaciones. Como debemos utilizar un multiplexor para seleccionar el resultado que nos interesa, no nos queda más remedio que elegir uno de tamaño 8 a 1, ya que el número de entradas de datos del multiplexor es siempre una potencia de 2.

| $S_2$ | $S_1$ | $S_0$ | N.º de entrada de datos del multiplexor | Operación                                  |
|-------|-------|-------|-----------------------------------------|--------------------------------------------|
| 0     | 0     | 0     | 0                                       | $A \text{ OR } B$ (suma lógica)            |
| 0     | 0     | 1     | 1                                       | $A \text{ AND } B$ (producto lógico)       |
| 0     | 1     | 0     | 2                                       | $A \text{ XOR } B$ (suma lógica exclusiva) |
| 0     | 1     | 1     | 3                                       | $A + B$ (suma aritmética)                  |
| 1     | 0     | 0     | 4                                       | Ninguna                                    |
| 1     | 0     | 1     | 5                                       | $A \text{ NOR } B$ (suma lógica negada)    |
| 1     | 1     | 0     | 6                                       | Ninguna                                    |
| 1     | 1     | 1     | 7                                       | $A - B$ (resta aritmética)                 |

Para ahorrar componentes, las operaciones de suma y de resta aritméticas las haremos con el mismo sumador completo. Esto nos obliga a incluir un multiplexor para elegir entre  $B_i$  y  $\bar{B}_i$ .

Por el mismo motivo, la operación NOR la realizaremos con una puerta AND, ya que sabemos que  $A \text{ NOR } B = \overline{A + B} = \bar{A} \cdot \bar{B} = \bar{A} \text{ AND } \bar{B}$ . De esta forma, necesitamos otro multiplexor para elegir entre  $A_i$  y  $\bar{A}_i$ .

En cuanto a la señal de selección del multiplexor de  $A_i$ , si conectamos  $A_i$  a la entrada de peso 0 y  $\bar{A}_i$  a la de peso 1, tenemos lo siguiente:

| $S_2$ | $S_1$ | $S_0$ | Valor necesario | Señal de selección del multiplexor de $A_i$ |
|-------|-------|-------|-----------------|---------------------------------------------|
| 0     | 0     | 0     | —               | X                                           |
| 0     | 0     | 1     | $A_i$           | 0                                           |
| 0     | 1     | 0     | —               | X                                           |
| 0     | 1     | 1     | $A_i$           | 0                                           |
| 1     | 0     | 0     | —               | X                                           |
| 1     | 0     | 1     | $\bar{A}_i$     | 1                                           |
| 1     | 1     | 0     | —               | X                                           |
| 1     | 1     | 1     | $A_i$           | 0                                           |

| $S_2 \backslash S_1 S_0$ | 00 | 01 | 11 | 10 |
|--------------------------|----|----|----|----|
| 0                        | X  | 0  | 0  | X  |
| 1                        | X  | 1  | 0  | X  |

Claramente se ve por Karnaugh que esta señal vale  $S_2 \bar{S}_1$ .

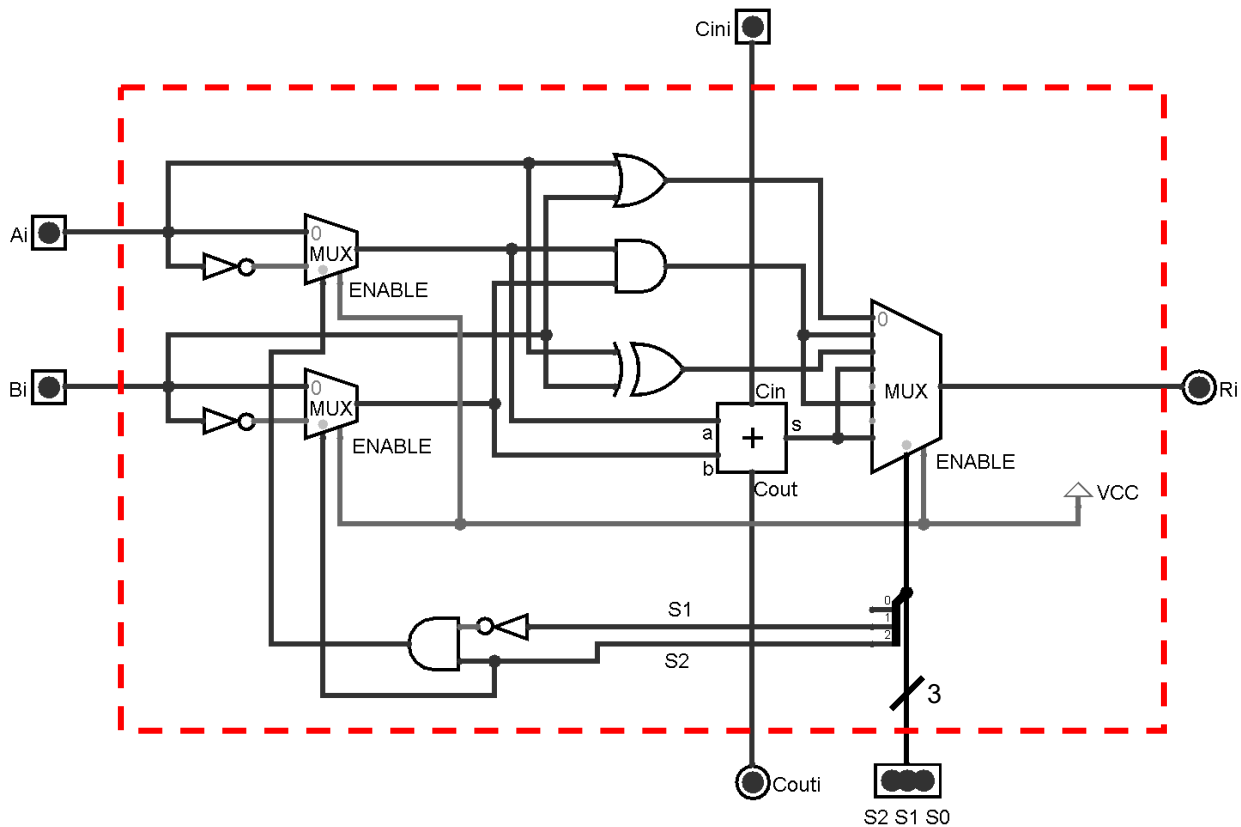
Por lo que respecta a la señal de selección del multiplexor de  $B_i$ , si conectamos  $B_i$  a la entrada de peso 0 y  $\bar{B}_i$  a la de peso 1, tenemos lo siguiente:

| $S_2$ | $S_1$ | $S_0$ | Valor necesario | Señal de selección del multiplexor de $B_i$ |
|-------|-------|-------|-----------------|---------------------------------------------|
| 0     | 0     | 0     | —               | X                                           |
| 0     | 0     | 1     | $B_i$           | 0                                           |
| 0     | 1     | 0     | —               | X                                           |
| 0     | 1     | 1     | $B_i$           | 0                                           |
| 1     | 0     | 0     | —               | X                                           |
| 1     | 0     | 1     | $\bar{B}_i$     | 1                                           |
| 1     | 1     | 0     | —               | X                                           |
| 1     | 1     | 1     | $\bar{B}_i$     | 1                                           |

| $S_2 \backslash S_1 S_0$ | 00 | 01 | 11 | 10 |
|--------------------------|----|----|----|----|
| 0                        | X  | 0  | 0  | X  |
| 1                        | X  | 1  | 1  | X  |

Simplificando, se ve por Karnaugh que esta señal de selección coincide exactamente con  $S_2$ .

Por lo demás, el resto de esta parte del ejercicio es similar al anterior. Por tanto, el diseño de la miniALU de peso  $i$  sería el siguiente ( $0 \leq i \leq 7$ ):



```

(bit Couti, bit ri) ALU_1_bit(bit ai, bit bi, bit Cini, bit s[3])
{
    bit enable = 1
    bit entradasMux2a1_a[2]
    entradasMux2a1_A[0] = ai
    entradasMux2a1_A[1] = NOT ai
    bit a = mux2a1(entradasMux2a1_A, s[2] AND (NOT s[1]), enable)
    bit entradasMux2a1_B[2]
    entradasMux2a1_B[0] = bi
    entradasMux2a1_B[1] = NOT bi
    bit b = mux2a1(entradasMux2a1_B, s[2], enable)
    bit Couti, S, ri
    (Couti, S) = sumadorCompleto_1_bit(a, b, Cini)
    bit entradasMux8a1_R[8]
    entradasMux8a1_R[0] = ai OR bi
    entradasMux8a1_R[1] = ai AND bi
    entradasMux8a1_R[2] = ai XOR bi
    entradasMux8a1_R[3] = S
    entradasMux8a1_R[4] = undefined
    entradasMux8a1_R[5] = ai AND bi
    entradasMux8a1_R[6] = undefined
    entradasMux8a1_R[7] = S
    ri = mux8a1(entradasMux8a1_R, S, enable)
    return (Couti, ri)
}

```

b) Ahora pasaremos a replicar la miniALU de 1 bit del apartado anterior un total de 8 veces, de tal forma que las encadenaremos en serie para la propagación de los acarrees. Dicho de otro modo, el acarreo de salida  $C_{out_i}$  se conectará con el acarreo de entrada  $C_{in_i}$ .

En cuanto al acarreo de entrada inicial  $C_{in_0}$ , debemos calcularlo para que tome el valor correcto para las operaciones de suma y de resta aritméticas.

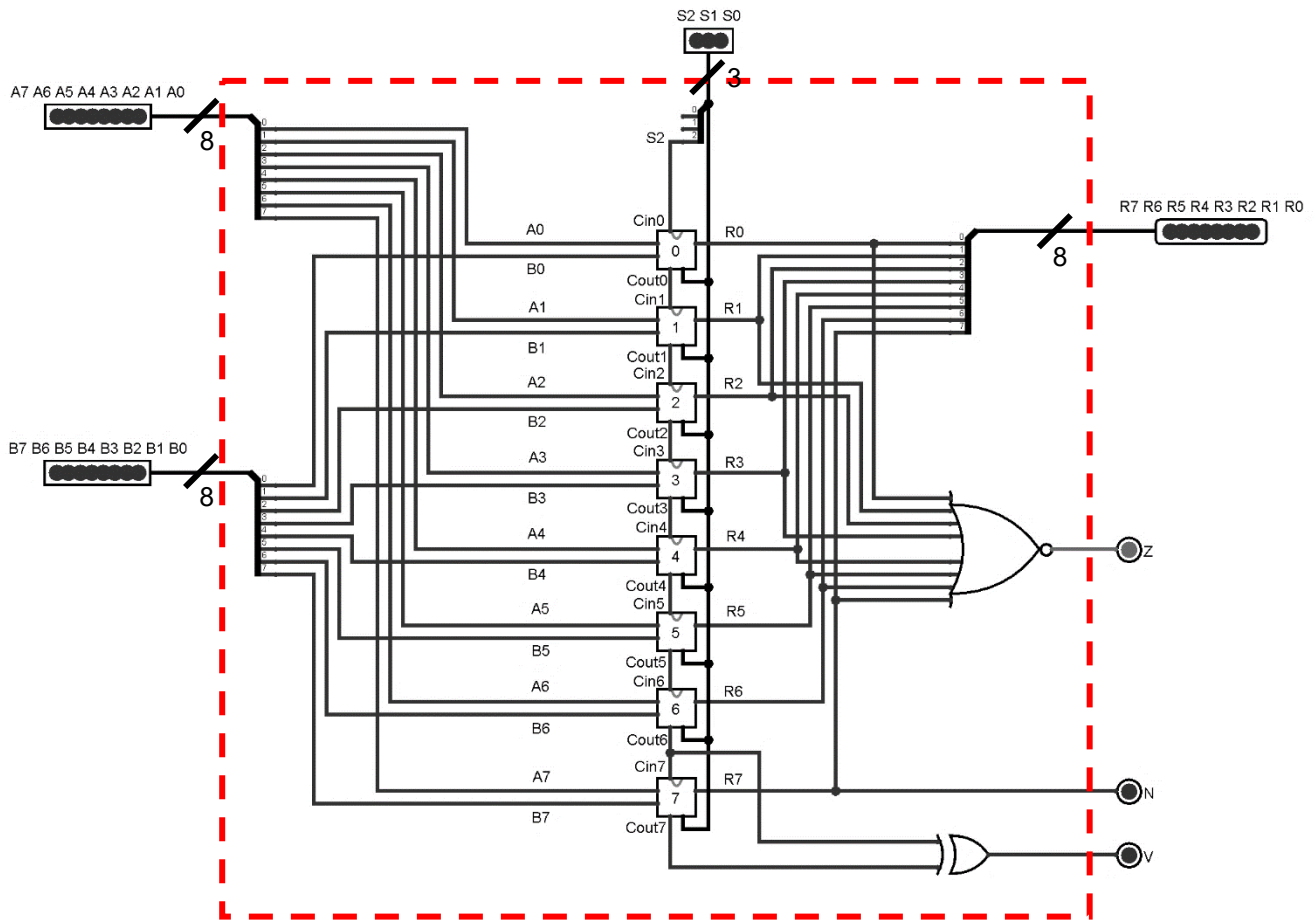
| $S_2$ | $S_1$ | $S_0$ | $C_{in_0}$ |
|-------|-------|-------|------------|
| 0     | 0     | 0     | X          |
| 0     | 0     | 1     | X          |
| 0     | 1     | 0     | X          |
| 0     | 1     | 1     | 0          |
| 1     | 0     | 0     | X          |
| 1     | 0     | 1     | X          |
| 1     | 1     | 0     | X          |
| 1     | 1     | 1     | 1          |

| $S_2 \backslash S_1 S_0$ | 00 | 01 | 11 | 10 |
|--------------------------|----|----|----|----|
| 0                        | X  | X  | 0  | X  |
| 1                        | X  | X  | 1  | X  |

Por tanto, según el mapa de Karnaugh,  $C_{in_0}$  vale  $S_2$ .

En cuanto a los indicadores de resultado, podemos hallarlos de la siguiente manera:

- El detector de ceros  $Z$  se calcula mediante una puerta NOR de 8 entradas:  $\overline{R_0 + R_1 + \dots + R_7}$ .
- El detector de resultado negativo  $N$  coincide con el bit más significativo del resultado ( $R_7$ ).
- En cuanto al detector de desbordamiento en complemento a 2, se puede implementar de dos maneras:
  - Comprobando que los signos de  $A_i$  y  $B_i$  son iguales (puerta XNOR) y, a la vez (puerta AND), el signo de  $A_i$  es contrario al de  $R_i$  (puerta XOR), es decir,  $(\overline{A_i \oplus B_i}) \cdot (A_i \oplus R_i)$ .
  - O bien, comprobando que los dos últimos acarrees de salida son bits contrarios (puerta XOR), es decir,  $C_{out_6} \oplus C_{out_7}$ . Por simplicidad, lo implementaremos de esta manera.



```

(bit r[8], bit z, bit n, bit v) ALU_8_bits(bit a[8], bit b[8], bit s[3])
{
    bit r[8]
    bit Cout[8]
    bit z, n, v
    (Cout[0], r[0]) = ALU_1_bit(a[0], b[0], s[2], s)
    z = r[0]
    for (int i = 1; i < 8; i++)
    {
        (Cout[i], r[i]) = ALU_1_bit(a[i], b[i], Cout[i-1], s)
        z = z OR r[i]
    }
    z = NOT z
    n = r[7]
    v = Cout[6] XOR Cout[7]
    return (r, z, n, v)
}

```

c) Para terminar, añadiremos más indicadores de resultado según nos pide el enunciado. Para realizar todas estas comparaciones, podríamos hacer la resta  $A - B$  ( $S_2S_1S_0 = 111$ ) y utilizar los indicadores  $Z$ ,  $N$  y  $V$ .

- Por ejemplo, al comprobar la igualdad, la resta será nula y se activará la señal  $Z$ . Por tanto, resulta evidente que  $EQ = Z$ .
- La desigualdad será justo lo contrario a la igualdad, con lo que  $NE = \overline{EQ} = \bar{Z}$ .
- En cuanto a la señal de menor, debemos recordar que estamos trabajando en complemento a 2. Deben cumplirse dos condiciones:
  - Que la resta sea negativa, es decir, que  $N = 1$ .
  - Y, a la vez, que no haya producido desbordamiento, es decir, que  $V = 0$ .

Por tanto, se tiene que  $LT = N \cdot \bar{V}$ .

- Para incluir la condición del igual al caso anterior, debemos considerar también el caso de que el resultado sea cero, con lo que  $LE = (N + Z) \cdot \bar{V}$ .
- El resto de las condiciones serían las complementarias, es decir,  $GT = \overline{LE}$  y  $GE = \overline{LT}$ .

Se deja como ejercicio incluir estas salidas al dibujo del circuito.

3. **[Ejercicio de examen]** Se desea diseñar una unidad aritmético-lógica (ALU) para datos de 4 bits ( $A = A_3A_2A_1A_0$  y  $B = B_3B_2B_1B_0$ ) expresados en complemento a 2, que realice las siguientes operaciones:

| $S_1$ | $S_0$ | Operación   |
|-------|-------|-------------|
| 0     | 0     | $A + B$     |
| 0     | 1     | $B - A$     |
| 1     | 0     | $A - B - 1$ |
| 1     | 1     | $A - B$     |

Además de los 4 bits del resultado ( $R = R_3R_2R_1R_0$ ), la ALU tendría otras 3 salidas de 1 bit:

- Bit  $N$ , que valdría 1 si el resultado de la operación es negativo.
- Bit  $Z$ , que valdría 1 si el resultado de la operación es cero.
- Bit  $P$ , que valdría 1 si el resultado es positivo (sin contar el cero).

Para resolver este ejercicio se ha de seguir un diseño modular de abajo arriba y se podrán utilizar puertas lógicas básicas, así como bloques combinacionales estándar, como multiplexores, sumadores completos de 1 bit, etc.

Se pide:

- Dibujar y explicar cómo sería una miniALU para datos de 1 bit que realizara las 4 operaciones posibles.
- Dibujar y explicar cómo habría que enlazar entre sí las 4 miniALUs para conseguir la ALU completa de 4 bits.

### Solución:

a) Empezando por la miniALU de 1 bit, las cuatro operaciones las vamos a realizar con un único sumador completo, ya que sabemos que suma 3 bits ( $a$ ,  $b$  y  $C_{in}$ ). Por tanto, como los datos  $A$  y  $B$  están en complemento a 2, podemos reescribir las operaciones como la suma de 3 términos:

| $S_1$ | $S_0$ | Operación equivalente                                                  | $a$         | $b$         | $C_{in0}$ |
|-------|-------|------------------------------------------------------------------------|-------------|-------------|-----------|
| 0     | 0     | $A + B + 0$                                                            | $A_i$       | $B_i$       | 0         |
| 0     | 1     | $-A + B =$<br>$= C_2(A) + B =$<br>$= \bar{A} + B + 1$                  | $\bar{A}_i$ | $B_i$       | 1         |
| 1     | 0     | $A + C_2(B) - 1 =$<br>$= A + \bar{B} + 1 - 1 =$<br>$= A + \bar{B} + 0$ | $A_i$       | $\bar{B}_i$ | 0         |
| 1     | 1     | $A + C_2(B) + 1 =$<br>$= A + \bar{B} + 1$                              | $A_i$       | $\bar{B}_i$ | 1         |

Para la miniALU de 1 bit, dejaremos libre el valor de  $C_{in}$ , ya que se establecerá en el nivel superior de diseño. Respecto a los operandos, usaremos sendos multiplexores de 2 entradas para seleccionar el valor adecuado. Las señales de selección de estos multiplexores las generaremos a partir de las entradas  $S_1$  y  $S_0$ .

En cuanto al primer operando del sumador completo ( $a$ ), conectaremos la señal  $A_i$  a la entrada 0 del multiplexor, mientras que la señal  $\bar{A}_i$  irá a la entrada 1.

| $S_1$ | $S_0$ | $a$         | Señal de selección del multiplexor de $A_i$ |
|-------|-------|-------------|---------------------------------------------|
| 0     | 0     | $A_i$       | 0                                           |
| 0     | 1     | $\bar{A}_i$ | 1                                           |
| 1     | 0     | $A_i$       | 0                                           |
| 1     | 1     | $A_i$       | 0                                           |

| $S_1 \backslash S_0$ | 0 | 1 |
|----------------------|---|---|
| 0                    | 0 | 1 |
| 1                    | 0 | 0 |

Esta señal de control vale  $\bar{S}_1 S_0$ , según Karnaugh, ya que este producto solo vale 1 para la combinación 01.

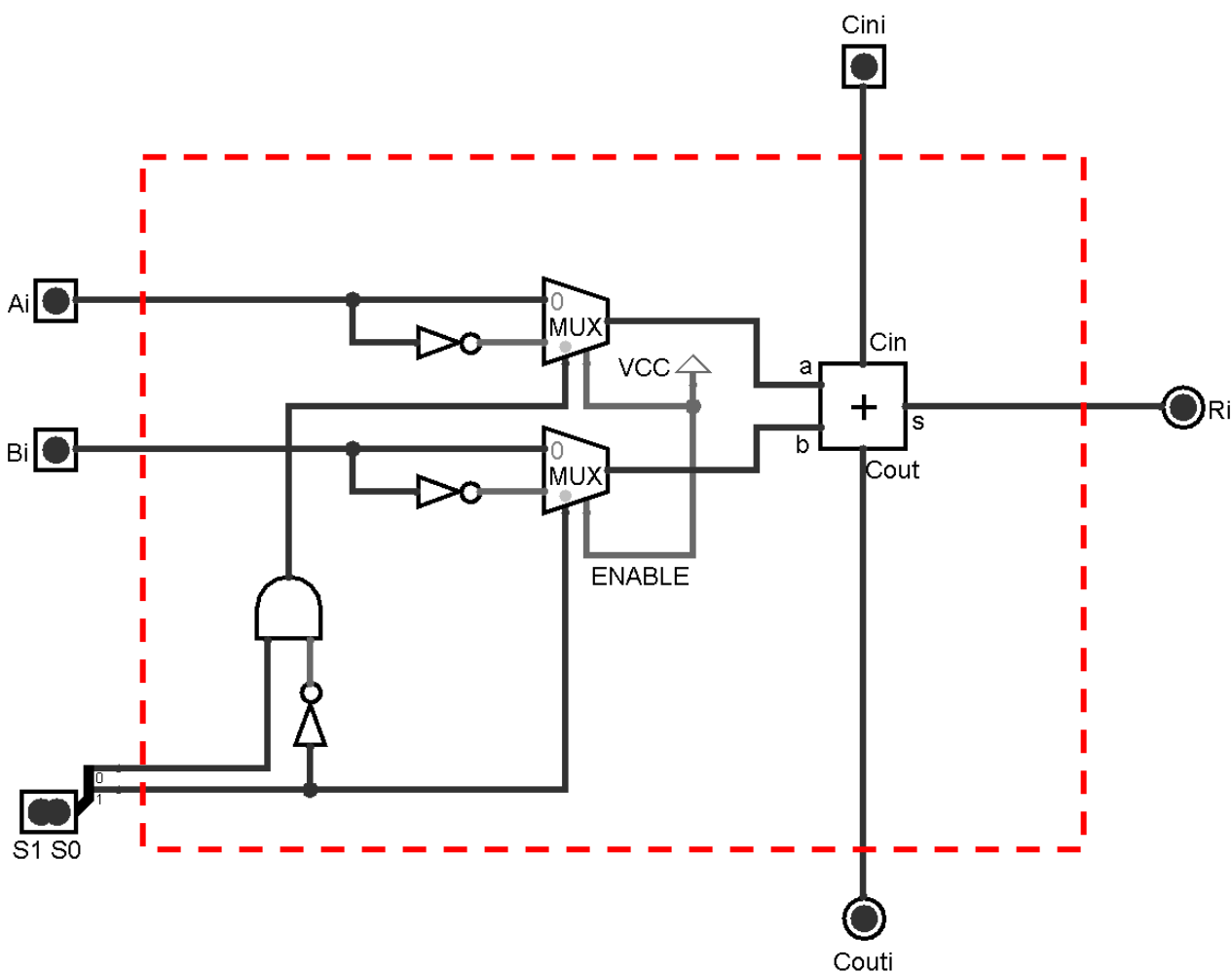
En cuanto al segundo operando del sumador completo ( $b$ ), conectaremos la señal  $B_i$  a la entrada 0 del multiplexor, mientras que la señal  $\bar{B}_i$  irá a la entrada 1.

| $S_1$ | $S_0$ | $b$         | Señal de selección del multiplexor de $B_i$ |
|-------|-------|-------------|---------------------------------------------|
| 0     | 0     | $B_i$       | 0                                           |
| 0     | 1     | $B_i$       | 0                                           |
| 1     | 0     | $\bar{B}_i$ | 1                                           |
| 1     | 1     | $\bar{B}_i$ | 1                                           |

| $S_1 \backslash S_0$ | 0 | 1 |
|----------------------|---|---|
| 0                    | 0 | 0 |
| 1                    | 1 | 1 |

Esta señal de control vale exactamente  $S_1$  según Karnaugh.

Por tanto, el diseño de la miniALU de 1 bit sería el siguiente ( $0 \leq i \leq 3$ ):



Nótese que, como el sumador completo calcula las 4 operaciones posibles, no se necesita un multiplexor para seleccionar el valor correcto de  $R_i$ .

```
(bit Couti, bit ri) ALU_1_bit(bit ai, bit bi, bit cini, bit s[2])
{
    bit enable = 1
    bit entradasMux2a1_A[2]
    entradasMux2a1_A[0] = ai
    entradasMux2a1_A[1] = NOT ai
    bit a = mux2a1(entradasMux2a1_A, s[0] AND (NOT s[1]), enable)
    bit entradasMux2a1_B[2]
    entradasMux2a1_B[0] = bi
    entradasMux2a1_B[1] = NOT bi
    bit b = mux2a1(entradasMux2a1_B, s[1], enable)
    bit Couti, ri
    (Couti, ri) = sumadorCompleto_1_bit(a, b, cini)
    return (Couti, ri)
}
```

b) La ALU anterior se repite 4 veces, donde la  $ALU_i$  recibe los bits  $A_i$  y  $B_i$ . El acarreo de salida  $C_{out_i}$  se propaga a la siguiente ALU, es decir, coincide con  $C_{in_{i+1}}$ , con la excepción del último acarreo  $C_{out_3}$ , que se pierde porque no forma parte del resultado al estar trabajando con datos en complemento a 2.

Por otro lado, el acarreo de entrada inicial  $C_{in_0}$  podemos calcularlo mediante la siguiente tabla:

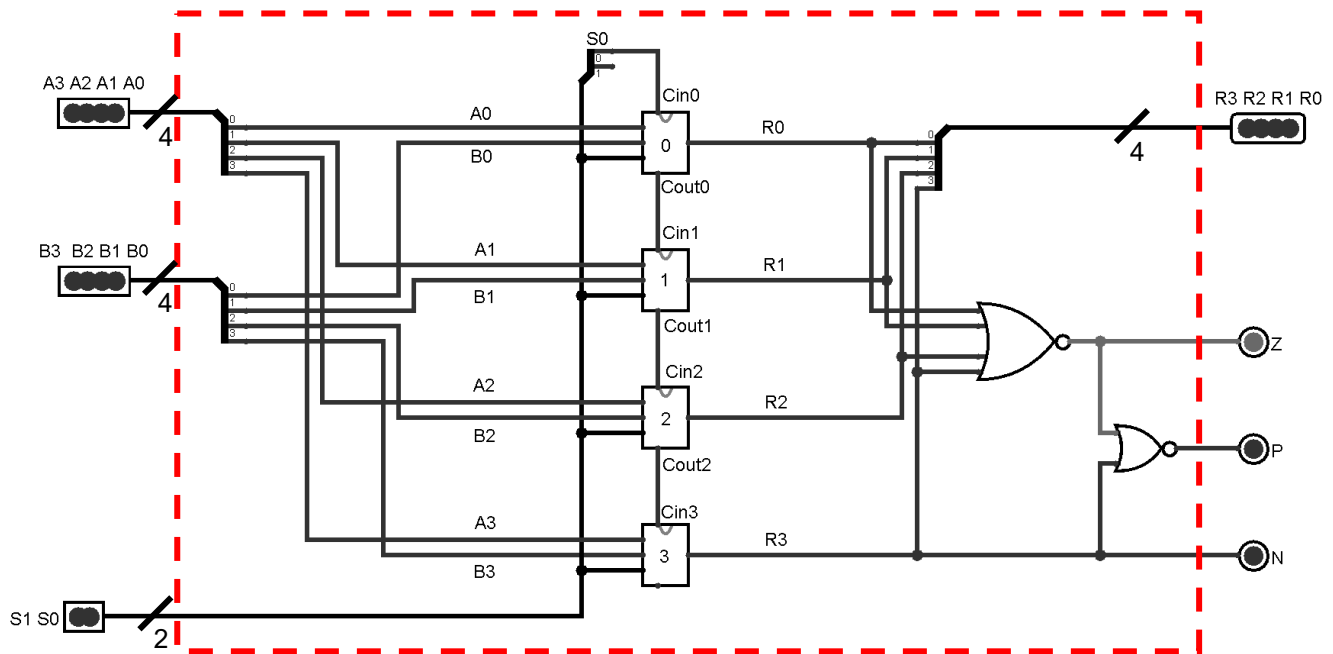
| $S_1$ | $S_0$ | $C_{in_0}$ |
|-------|-------|------------|
| 0     | 0     | 0          |
| 0     | 1     | 1          |
| 1     | 0     | 0          |
| 1     | 1     | 1          |

| $S_1 \backslash S_0$ | 0 | 1 |
|----------------------|---|---|
| 0                    | 0 | 1 |
| 1                    | 0 | 1 |

Por tanto, esta señal es igual a  $S_0$ , según el mapa de Karnaugh.

En cuanto a los indicadores de resultado, se tiene que:

- $N = R_3$ , ya que el bit de signo es el más significativo del resultado.
- $Z = \overline{R_3 + R_2 + R_1 + R_0}$ , lo que se consigue con una puerta NOR de 4 entradas con todos los bits del resultado.
- Para hallar  $P$ , debemos recordar que, si el resultado NO es negativo Y NO es cero, tiene que ser positivo a la fuerza. Luego  $P = \overline{N} \overline{Z} = \overline{N} + \overline{Z}$ . Podemos hallarlo, por tanto, mediante una puerta AND de 2 entradas más dos puertas NOT o, mejor aún, mediante una única puerta NOR de 2 entradas.



```

(bit r[4], bit z, bit p, bit n) ALU_4_bits(bit a[4], bit b[4], bit s[2])
{
    bit r[4]
    bit z, p, n
    bit Cout[4]
    (Cout[0], r[0]) = ALU_1_bit(a[0], b[0], s[0], s)
    z = r[0]
    for (int i = 1; i < 4; i++)
    {
        (Cout[i], r[i]) = ALU_1_bit(a[i], b[i], Cout[i-1], s)
        z = z OR r[i]
    }
    z = NOT z
    n = r[3]
    p = z NOR n
    return (r, z, p, n)
}

```

4. **[Ejercicio de examen]** Se pide diseñar razonadamente y dibujar una unidad aritmético-lógica (ALU) para 2 operandos ( $A = A_3A_2A_1A_0$  y  $B = B_3B_2B_1B_0$ ), que se encuentran expresados en complemento a 2. Las operaciones permitidas son las siguientes:

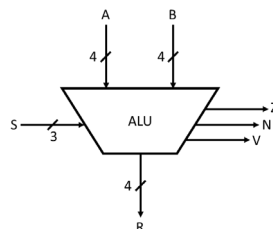
| $S_2$ | $S_1$ | $S_0$ | Operación                  |
|-------|-------|-------|----------------------------|
| 0     | 0     | 0     | $A \text{ OR } B$          |
| 0     | 0     | 1     | $A + B$ (suma aritmética)  |
| 0     | 1     | 0     | $\text{NOT } A$            |
| 0     | 1     | 1     | Sin operación (devuelve 0) |
| 1     | 0     | 0     | $A - B$ (resta aritmética) |
| 1     | 0     | 1     | $A \text{ AND } B$         |
| 1     | 1     | 0     | $A \text{ NOR } B$         |
| 1     | 1     | 1     | $\text{NOT } B$            |

Para realizar este ejercicio se deberán tener en cuenta las siguientes consideraciones:

- El diseño debe ser modular, con lo que habrá que diseñar primero una ALU básica para operandos de 1 bit y particularizarla según la posición que ocupe dicho bit en los operandos  $A$  y  $B$ .
- Se podrán utilizar puertas lógicas auxiliares del tamaño necesario.
- Se podrán utilizar bloques combinacionales auxiliares del tamaño necesario, como decodificadores, multiplexores, comparadores, etc.
- La suma y resta aritméticas deben realizarse obligatoriamente a partir de sumadores completos de 1 bit.
- Habrá que indicar el valor de las señales de control auxiliares.

Las salidas de la ALU son las siguientes:

- Resultado  $R = R_3R_2R_1R_0$ .
- Bit de resultado nulo  $Z$ .
- Bit de resultado negativo  $N$ .
- Bit de desbordamiento  $V$  (según corresponda en la suma y resta aritméticas).



### Solución:

Usaremos, como es habitual, un diseño jerárquico y modular para la ALU. Cada bit se maneja con una miniALU básica y luego se conectarán entre sí para trabajar con los datos de 4 bits. Si algún bit tiene un trato especial, habrá que diseñar una ALU específica para ese caso. En nuestro caso, usaremos una miniALU de 1 bit común, pero algunas de sus entradas o salidas serán tratadas de manera diferente.

Para la miniALU de peso  $i$  ( $0 \leq i \leq 3$ ), hay 7 operaciones, con lo que usaremos un multiplexor de 8 a 1 para seleccionar el resultado adecuado. A cada entrada del multiplexor colocaremos el resultado que corresponde a cada operación. Como este multiplexor funciona siempre, su entrada de *ENABLE* estará conectada a la fuente de alimentación  $V_{CC}$  (valor lógico 1). Las entradas son las siguientes:

- **Entrada 0:** Recibe el resultado obtenido con una puerta OR2, que realiza la operación  $A_i \text{ OR } B_i$ .
- **Entrada 1:** Aquí conectaremos la salida de la suma de un sumador completo de 1 bit. Al tratarse de una suma, debemos establecer que el acarreo de entrada  $C_{in_0}$  valga 0 (esto se realiza desde fuera de la miniALU).
- **Entrada 2:** Conectaremos una puerta NOT para hallar el valor  $\bar{A}_i$ . Aquí se ignora el bit  $B_i$ .
- **Entrada 3:** No realiza ninguna operación y devuelve el valor constante 0. Para ello, cablearemos esta entrada a tierra *GND*.
- **Entrada 4:** Debemos realizar una resta, para lo cual conectaremos también la salida de la suma del mismo sumador completo de la entrada 1. Para que la resta se calcule correctamente en complemento a 2, debemos imponer que el acarreo de entrada  $C_{in_0}$  valga 1, mientras que el segundo operando del sumador debe ser  $\bar{B}_i$ . Para poder elegir entre  $B_i$  y  $\bar{B}_i$ , necesitamos añadir un multiplexor 2 a 1. Más abajo calcularemos la señal de selección de este multiplexor.
- **Entrada 5:** Aquí colocamos una puerta AND2, que calcula la operación  $A_i \text{ AND } B_i$ .
- **Entrada 6:** Necesitamos ahora una puerta NOR2 para hallar la suma negada. Si disponemos de ella, la colocaríamos aquí y ya está. Sin embargo, como nos piden reaprovechar los componentes en nuestro diseño en la medida de lo posible, vamos a reaprovechar la puerta AND2 de la entrada 5. En este caso, podemos reescribir la operación  $\bar{A}_i \text{ OR } \bar{B}_i$  como  $\bar{A}_i \text{ AND } \bar{B}_i$ . Para poder elegir entre  $A_i$  y  $\bar{A}_i$ , necesitamos añadir otro multiplexor 2 a 1. Más abajo calcularemos la señal de selección de este multiplexor.
- **Entrada 7:** Conectaremos una puerta NOT para hallar el valor  $\bar{B}_i$ . Aquí se ignora el bit  $A_i$ .

Veamos ahora las señales de selección de los multiplexores. En cuanto al que selecciona entre  $A_i$  y  $\bar{A}_i$ , si colocamos  $A_i$  en la entrada de peso 0 y  $\bar{A}_i$  en la de peso 1, tenemos:

| $S_2$ | $S_1$ | $S_0$ | Operación                  | Operando necesario | Señal |
|-------|-------|-------|----------------------------|--------------------|-------|
| 0     | 0     | 0     | $A \text{ OR } B$          | —                  | X     |
| 0     | 0     | 1     | $A + B$ (suma aritmética)  | $A_i$              | 0     |
| 0     | 1     | 0     | NOT $A$                    | —                  | X     |
| 0     | 1     | 1     | Sin operación (devuelve 0) | —                  | X     |
| 1     | 0     | 0     | $A - B$ (resta aritmética) | $A_i$              | 0     |
| 1     | 0     | 1     | $A \text{ AND } B$         | $A_i$              | 0     |
| 1     | 1     | 0     | $A \text{ NOR } B$         | $\bar{A}_i$        | 1     |
| 1     | 1     | 1     | NOT $B$                    | —                  | X     |

| $S_2 \backslash S_1 S_0$ | 00 | 01 | 11 | 10 |
|--------------------------|----|----|----|----|
| 0                        | X  | 0  | X  | X  |
| 1                        | 0  | 0  | X  | 1  |

Por tanto, esta señal valdrá exactamente igual que  $S_1$ , según Karnaugh.

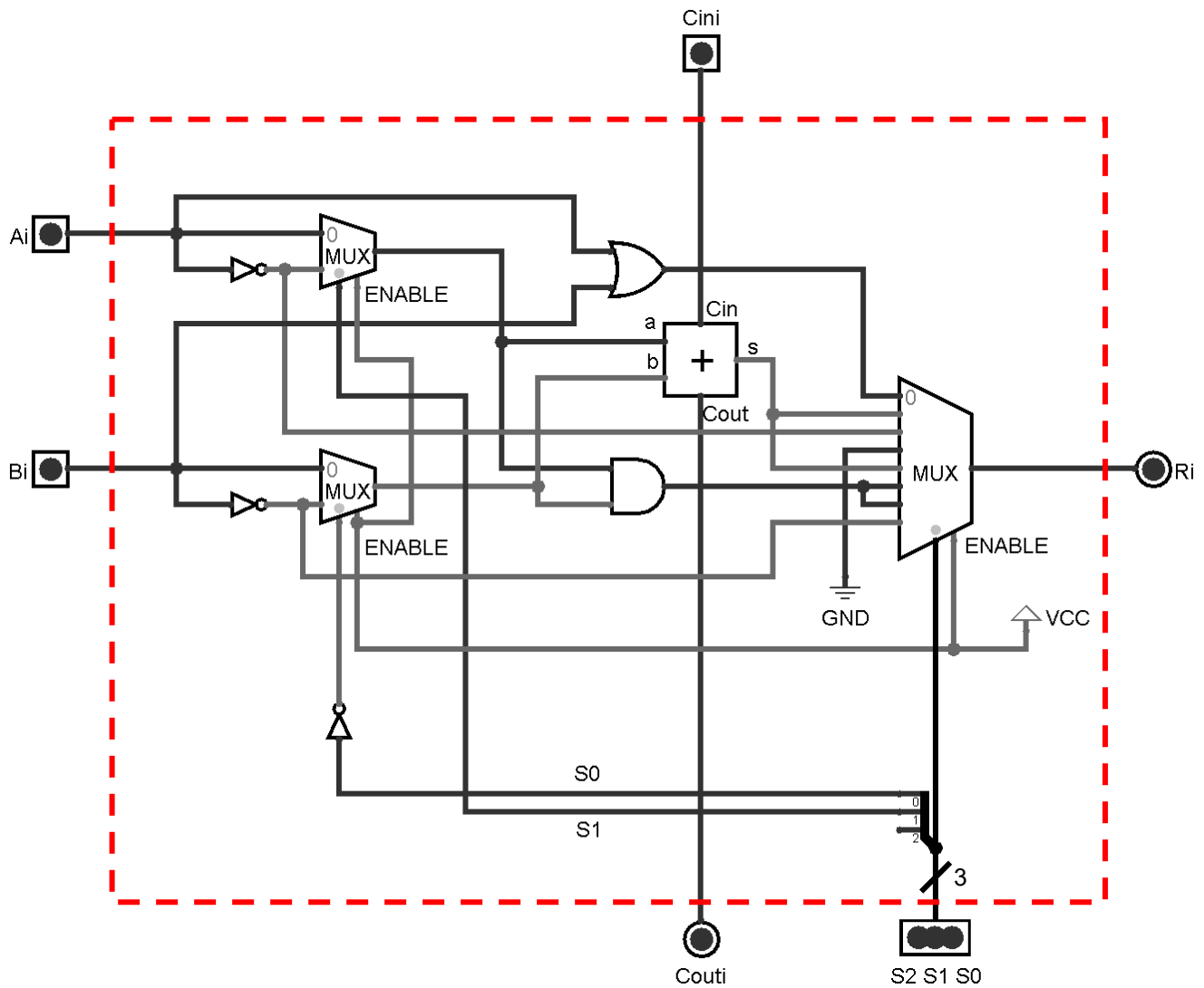
En cuanto al que selecciona entre  $B_i$  y  $\bar{B}_i$ , si colocamos  $B_i$  en la entrada de peso 0 y  $\bar{B}_i$  en la de peso 1, tenemos:

| $S_2$ | $S_1$ | $S_0$ | Operación                  | Operando necesario | Señal |
|-------|-------|-------|----------------------------|--------------------|-------|
| 0     | 0     | 0     | $A \text{ OR } B$          | —                  | X     |
| 0     | 0     | 1     | $A + B$ (suma aritmética)  | $B_i$              | 0     |
| 0     | 1     | 0     | NOT $A$                    | —                  | X     |
| 0     | 1     | 1     | Sin operación (devuelve 0) | —                  | X     |
| 1     | 0     | 0     | $A - B$ (resta aritmética) | $\bar{B}_i$        | 1     |
| 1     | 0     | 1     | $A \text{ AND } B$         | $B_i$              | 0     |
| 1     | 1     | 0     | $A \text{ NOR } B$         | $\bar{B}_i$        | 1     |
| 1     | 1     | 1     | NOT $B$                    | —                  | X     |

| $S_2 \backslash S_1 S_0$ | 00 | 01 | 11 | 10 |
|--------------------------|----|----|----|----|
| 0                        | X  | 0  | X  | X  |
| 1                        | 1  | 0  | X  | 1  |

Por tanto, esta señal valdrá exactamente igual que  $\bar{S}_0$ , según Karnaugh.

El diseño de la miniALU de 1 bit sería el siguiente:



```

(bit Couti, bit ri) ALU_1_bit(bit ai, bit bi, bit cini, bit s[3])
{
    bit enable = 1
    bit entradasMux2a1_A[2]
    entradasMux2a1_A[0] = ai
    entradasMux2a1_A[1] = NOT ai
    bit a = mux2a1(entradasMux2a1_A, s[1], enable)
    bit entradasMux2a1_B[2]
    entradasMux2a1_B[0] = bi
    entradasMux2a1_B[1] = NOT bi
    bit b = mux2a1(entradasMux2a1_B, NOT s[0], enable)
    bit Couti, S, ri
    (Couti, s) = sumadorCompleto_1_bit(a, b, cini)
    bit entradasMux8a1_R[8]
    entradasMux8a1_R[0] = ai OR bi
    entradasMux8a1_R[1] = s
    entradasMux8a1_R[2] = NOT ai
}

```

```

entradasMux8a1_R[3] = 0
entradasMux8a1_R[4] = s
entradasMux8a1_R[5] = a AND b
entradasMux8a1_R[6] = a AND b
entradasMux8a1_R[7] = NOT bi
ri = mux8a1(entradasMux8a1_R, s, enable)
return (couti, ri)

```

```

}

```

Para la ALU completa de 4 bits, colocamos las 4 miniALUs en serie para propagar el acarreo en las sumas y las restas, de forma que cada acarreo de salida  $C_{out_i}$  está conectado al acarreo de entrada  $C_{in_{i+1}}$ , como es habitual. Por otro lado, el acarreo de entrada inicial va aparte. Calculemos su valor:

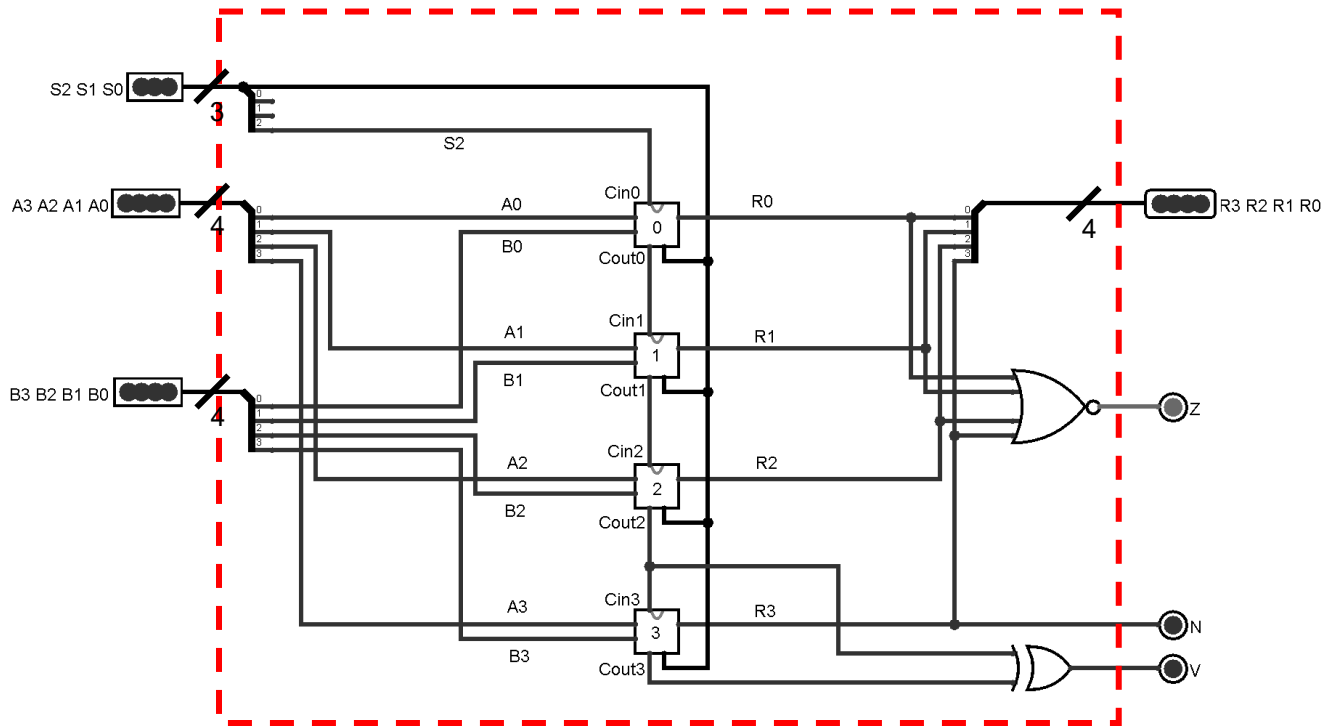
| $S_2$ | $S_1$ | $S_0$ | Operación                  | $C_{in_0}$ |
|-------|-------|-------|----------------------------|------------|
| 0     | 0     | 0     | $A \text{ OR } B$          | X          |
| 0     | 0     | 1     | $A + B$ (suma aritmética)  | 0          |
| 0     | 1     | 0     | NOT $A$                    | X          |
| 0     | 1     | 1     | Sin operación (devuelve 0) | X          |
| 1     | 0     | 0     | $A - B$ (resta aritmética) | 1          |
| 1     | 0     | 1     | $A \text{ AND } B$         | X          |
| 1     | 1     | 0     | $A \text{ NOR } B$         | X          |
| 1     | 1     | 1     | NOT $B$                    | X          |

|   | 00 | 01 | 11 | 10 |
|---|----|----|----|----|
| 0 | X  | 0  | X  | X  |
| 1 | 1  | X  | X  | X  |

Por tanto,  $C_{in_0}$  valdrá lo mismo que  $S_2$ , según Karnaugh.

En cuanto al detector de resultado nulo, usaremos una puerta NOR de 4 entradas con los bits del resultado. El detector de negativo coincidirá con el bit  $R_3$  del resultado. Y, por último, para el detector de desbordamiento en sumas y restas en complemento a 2, comprobaremos la desigualdad entre los dos últimos acarreos de salida, es decir,  $C_{out_3} \oplus C_{out_2}$ .

El esquema de la ALU completa sería el siguiente:



```

(bit r[4], bit z, bit n, bit v) ALU_4_bit(bit a[4], bit b[4], bit s[3])
{
    bit r[4]
    bit z, n, v
    bit Cout[4]
    (Cout[0], r[0]) = ALU_1_bit(a[0], b[0], s[2], s)
    z = r[0]
    for (int i = 1; i < 4; i++)
    {
        (Cout[i], r[i]) = ALU_1_bit(a[i], b[i], Cout[i-1], s)
        z = z OR r[i]
    }
    z = NOT z
    n = r[3]
    v = Cout[2] XOR Cout[3]
    return (r, z, n, v)
}

```

5. Se desea diseñar una unidad aritmético-lógica para datos de 4 bits,  $A$  ( $A_3A_2A_1A_0$ ) y  $B$  ( $B_3B_2B_1B_0$ ), que realice las siguientes operaciones según la señal de control  $S$  (3 bits):

| $S_2$ | $S_1$ | $S_0$ | Operación     |                 |
|-------|-------|-------|---------------|-----------------|
| 0     | 0     | 0     | $\bar{A}$     | Lógica          |
| 0     | 0     | 1     | $\bar{A} + B$ |                 |
| 0     | 1     | 0     | $A \cdot B$   |                 |
| 0     | 1     | 1     | $A \oplus B$  |                 |
| 1     | 0     | 0     | $A + B + 1$   | Aritmética (C2) |
| 1     | 0     | 1     | $A - B - 1$   |                 |
| 1     | 1     | 0     | $-A - 1$      |                 |
| 1     | 1     | 1     | $-B + 1$      |                 |

La ALU proporciona un resultado  $R$  ( $R_3R_2R_1R_0$ ), también de 4 bits.

Hay que realizar el diseño de la ALU en un proceso de arriba abajo de cuatro fases:

- En el **nivel 4**, el superior, disponemos de la ALU completa, vista desde el resto del computador. Recibe como entradas los operandos  $A$  y  $B$ , cada uno de 4 bits, así como la señal de control  $S$ , de 3 bits. Como salida, calcula el resultado  $R$ , de 4 bits.
- En el **nivel 3**, veremos que en realidad la ALU se compone de dos circuitos separados o semiALUs para datos de 4 bits, una de ellas especializada en las operaciones lógicas y la otra, en las aritméticas. En función del valor de  $S$ , se selecciona el resultado solicitado, el lógico o el aritmético.
- En el **nivel 2**, se ve la estructura interna de cada semiALU. En particular, se construirá mediante la repetición y conexión, en su caso, de las miniALUs para datos de 1 bit del nivel 1. El diseño de la semiALU aritmética será diferente, por razones evidentes, del de la semiALU lógica.
- En el **nivel 1**, el inferior, habrá que diseñar una miniALU para datos de 1 bit que realice las cuatro operaciones lógicas. Por otro lado, habrá que crear una segunda miniALU diferenciada de la anterior que realice las cuatro operaciones aritméticas. En ambos casos, las entradas de cada miniALU serán 1 bit de  $A$ , 1 bit de  $B$ , así como los bits de selección  $S$  que se crean necesarios. Como salida, cada miniALU proporciona 1 bit de  $R$ .

Cada vez que bajamos un nivel, estamos aplicando un zum para ver sus componentes con más detalle.

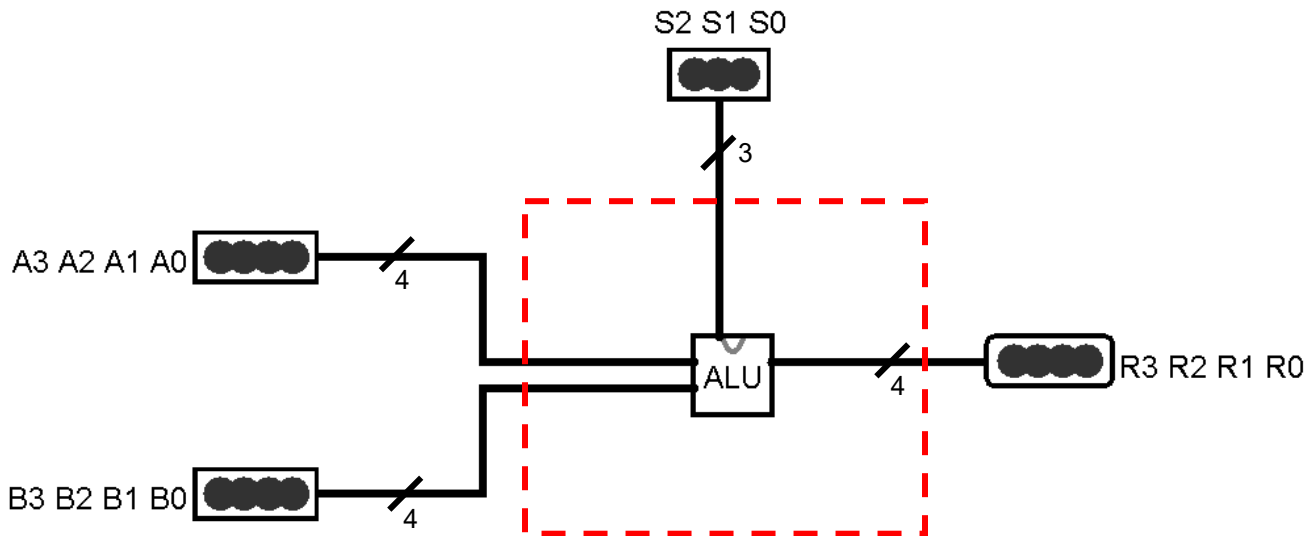
En cada nivel se deberán usar los componentes de niveles inferiores a modo de **caja negra**, es decir, sin mostrar su estructura interna, pero sí indicando explícitamente las entradas y las salidas. Para facilitar la corrección del ejercicio, cada caja negra deberá ir convenientemente nombrada y numerada y deberán indicarse sus entradas y sus salidas.

Para realizar este ejercicio se podrán usar todo tipo de puertas lógicas básicas, así como bloques combinacionales básicos como multiplexores del tamaño adecuado y sumadores completos de 1 bit (como cajas negras). Habrá que dibujar y explicar cada uno de los niveles de diseño de la ALU. **Justificar todas las decisiones de diseño.**

### Solución:

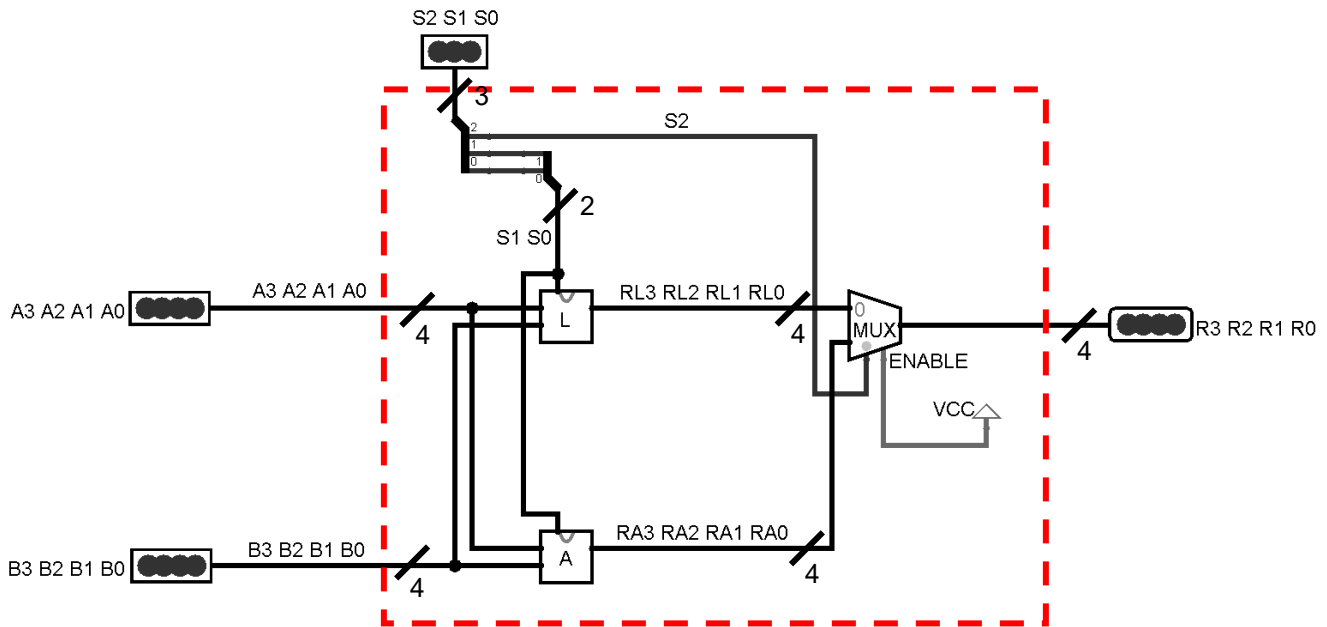
Comencemos de arriba abajo. El nivel 4 es el nivel en el que se ve la ALU desde el resto del computador. Por tanto, sin importarnos su estructura interna y a modo de caja negra, dispone de sus entradas de 4 bits para sendos operandos, así como la entrada de 3 bits para indicar la operación, y la salida de 4 bits con el resultado.

Como hemos realizado el dibujo del circuito con Logisim, el símbolo de la ALU es un cuadrado, pero, si lo hubiéramos dibujado a mano, habríamos puesto el típico símbolo con forma de V, donde los operandos entrarían por los extremos de la V, la señal de control de la operación entraría de manera lateral y el resultado aparecería por la parte picuda.



```
(bit r[4]) ALU_nivel4(bit a[4], bit b[4], bit s[3])
{
    bit r[4]
    r = ALU_nivel3(a, b, s)
    return (r)
}
```

Bajando al nivel 3, nos damos cuenta de que la ALU se compone en realidad de dos mitades, que hemos llamado semiALUs. Cada una de ellas recibe los dos operandos de 4 bits. En esta ocasión, la señal de operación es de solo 2 bits, no 3, ya que cada semiALU realiza 4 operaciones. Por otra parte, la salida de cada semiALU es un resultado de 4 bits, que hemos llamado resultado lógico *RL* y resultado aritmético *RA*, respectivamente. Como nos encontramos con dos posibles resultados simultáneamente, debemos colocar un multiplexor de 2 a 1 para datos de 4 bits. La señal de selección será precisamente  $S_2$  si colocamos los bits del resultado lógico en la entrada 0 y los del resultado aritmético en la entrada 1.

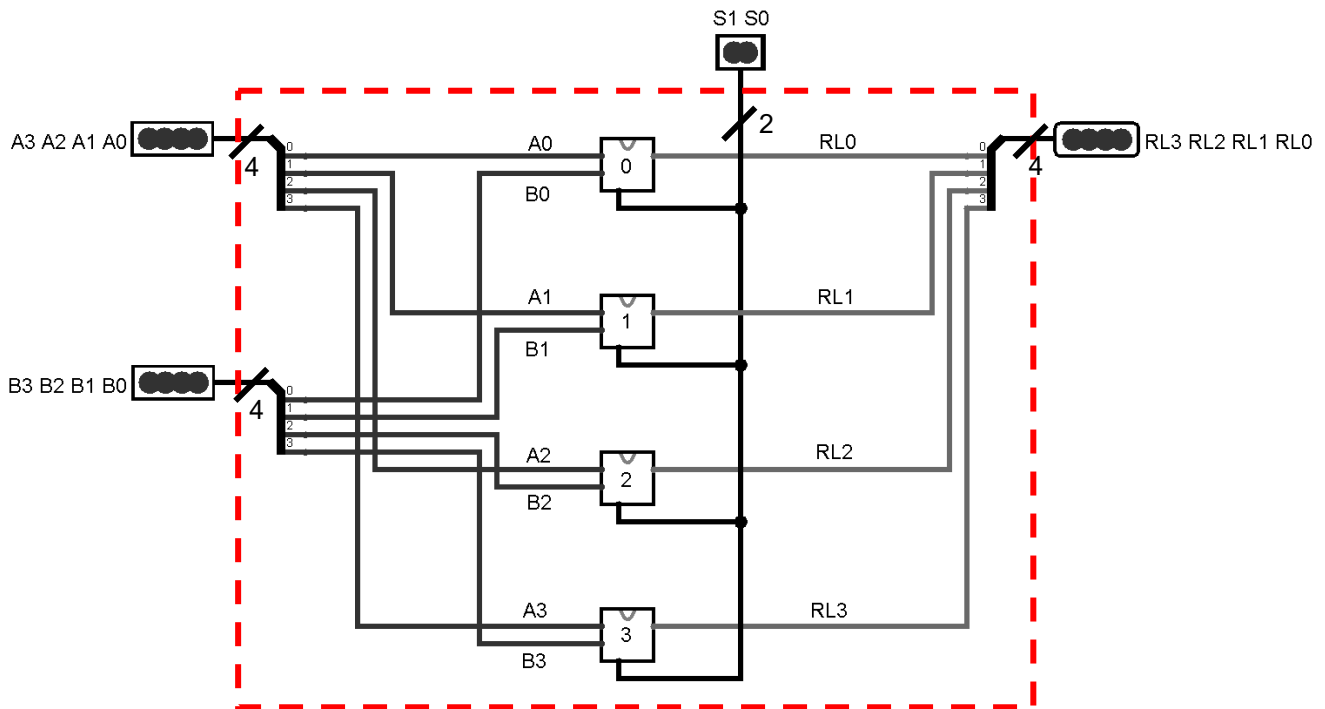


```

(bit r[4]) ALU_nivel3(bit a[4], bit b[4], bit s[3])
{
    bit enable = 1
    bit r[4]
    bit rl[4] // Resultado de la semiALU lógica
    bit ra[4] // Resultado de la semiALU aritmética
    rl = ALU_logica_nivel2(a, b, s[1] || s[0])
    ra = ALU_aritmetica_nivel2(a, b, s[1] || s[0])
    bit entradasMux2a1[2]
    for (int i = 0; i<4; i++)
    {
        entradasMux2a1[0] = rl[i]
        entradasMux2a1[1] = ra[i]
        r[i] = mux2a1(entradasMux2a1, s[2], enable)
    }
    return (r)
}

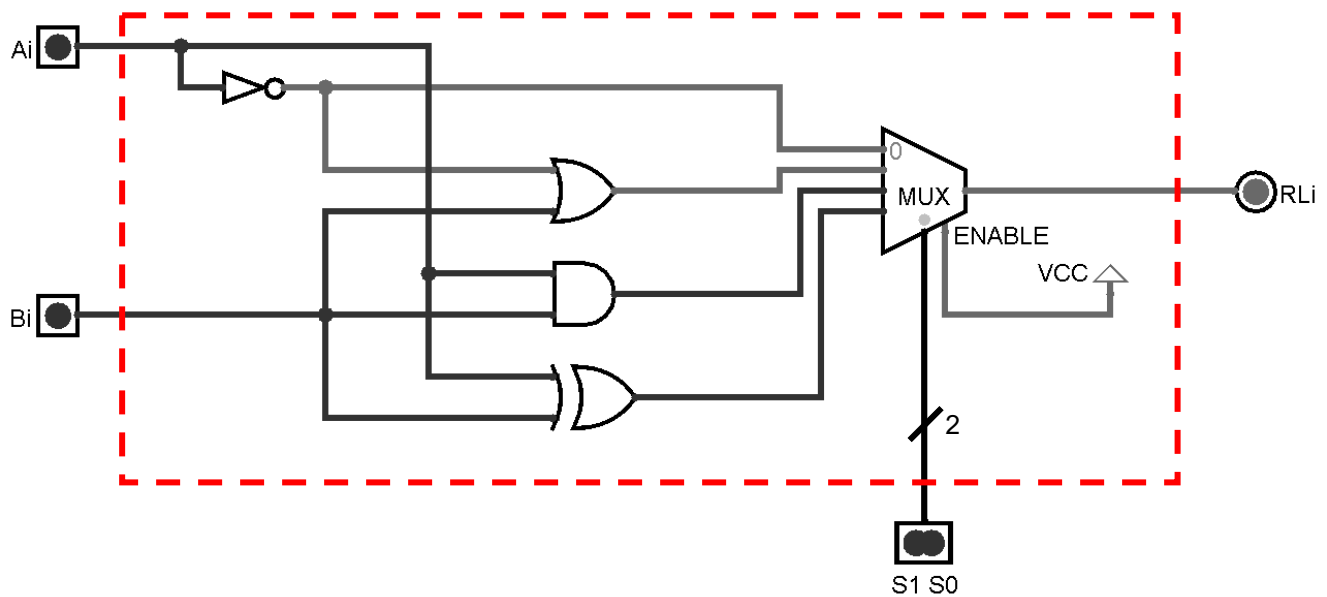
```

Llegando al nivel 2, ahora habrá dos tipos de diseño, uno por cada tipo de semiALU. Comenzando por la de tipo lógico, puede verse que este nivel está construido por la replicación 4 veces de la miniALU del nivel 1. Como en las operaciones lógicas no existe la propagación del acarreo, cada miniALU se encarga de cada uno de los bits de los operandos, sin que exista conexión entre las miniALUs de peso diferente.



El nivel 1 lógico se construye recibiendo sendos bits de los operandos, así como la señal de selección de 2 bits  $S_1 S_0$ . Calculamos los 4 posibles resultados con las puertas lógicas adecuadas y después elegimos el resultado adecuado con un multiplexor de 4 a 1, controlado precisamente por  $S_1 S_0$ .

```
(bit rl[4]) ALU_logica_nivel2(bit a[4], bit b[4], bit s[2])
{
    bit rl[4]
    for (int i = 0; i<4; i++)
    {
        rl[i] = ALU_logica_nivel1(a[i], b[i], s)
    }
    return (rl)
}
```



```

(bit rLi) ALU_logica_nivel1(bit ai, bit bi, bit s[2])
{
    bit enable = 1
    bit rLi
    bit entradasMux4a1_R[4]
    entradasMux4a1_R[0] = NOT ai
    entradasMux4a1_R[1] = (NOT ai) OR bi
    entradasMux4a1_R[2] = ai AND bi
    entradasMux4a1_R[3] = ai XOR bi
    rLi = mux4a1(entradasMux4a1_R, s, enable)
    return (rLi)
}

```

Pasemos ahora a la parte aritmética. Recordando que los operandos están expresados en complemento a 2, podemos reescribir las operaciones como:

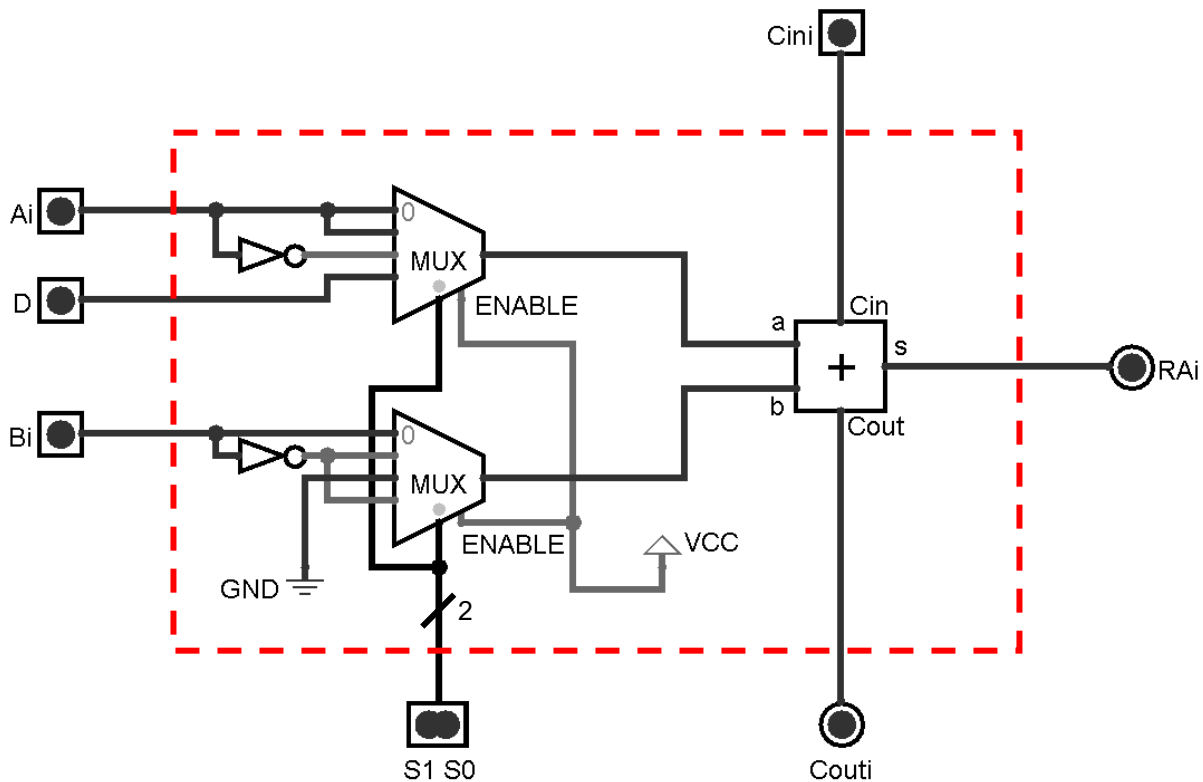
| $S_2$ | $S_1$ | $S_0$ | Operación                                                         |
|-------|-------|-------|-------------------------------------------------------------------|
| 1     | 0     | 0     | $A + B + 1$                                                       |
| 1     | 0     | 1     | $A - B - 1 = A + \bar{B} + 1 - 1 = A + \bar{B} = A + \bar{B} + 0$ |
| 1     | 1     | 0     | $-A - 1 = \bar{A} + 1 - 1 = \bar{A} + 0 + 0$                      |
| 1     | 1     | 1     | $-B + 1 = 1 + \bar{B} + 1$                                        |

Pensando en el nivel 1 aritmético, realizaremos las 4 operaciones con el mismo sumador completo de 1 bit, al que proporcionaremos el valor adecuado de los operandos. No será necesario usar un multiplexor 4 a 1 para seleccionar el resultado, ya que será directamente el bit s del sumador.

Por un lado, usaremos un multiplexor 4 a 1 para proporcionar el valor correcto del operando  $b$  del sumador, que puede ser  $B_i$ ,  $\bar{B}_i$ , 0 o  $\bar{B}_i$ , según la combinación de  $S_1 S_0$ .

Con respecto al valor correcto del operando  $a$  del sumador, puede ser  $A_i$ ,  $A_i$  y  $\bar{A}_i$  para las 3 primeras combinaciones de  $S_1 S_0$ . ¿Qué ocurre en la cuarta? Pues que deberíamos sumar 1, pero no a cada bit, lo cual daría la suma de 1111, es decir, 15 en base decimal, sino que solo deberíamos sumarlo al bit de peso 0, es decir, 0001. El valor 0 o 1 que debemos sumar lo vamos a establecer en el nivel 2 aritmético y aquí lo hemos llamado entrada  $D$ .

Por último, el acarreo de entrada  $C_{in_i}$  lo estableceremos también en el nivel 2 aritmético, con lo que también será una entrada en el nivel 1.



```
(bit Couti, bit ra_i) ALU_aritmetica_nivel1(bit a_i, bit b_i, bit Cini, bit d,
bit s[2])
{
    bit enable = 1
    bit Couti, ra_i
    bit a, b
    bit entradasMux4a1_a[4]
    entradasMux4a1_a[0] = a_i
    entradasMux4a1_a[1] = a_i
    entradasMux4a1_a[2] = NOT a_i
    entradasMux4a1_a[3] = d
    a = mux4a1(entradasMux4a1_a, s, enable)
```

```

bit entradasMux4a1_b[4]
entradasMux4a1_b[0] = bi
entradasMux4a1_b[1] = NOT bi
entradasMux4a1_b[2] = 0
entradasMux4a1_b[3] = NOT bi
b = mux4a1(entradasMux4a1_b, s, enable)
(couti, rai) = sumadorCompleto_1_bit(a, b, cini)
return (rai)
}

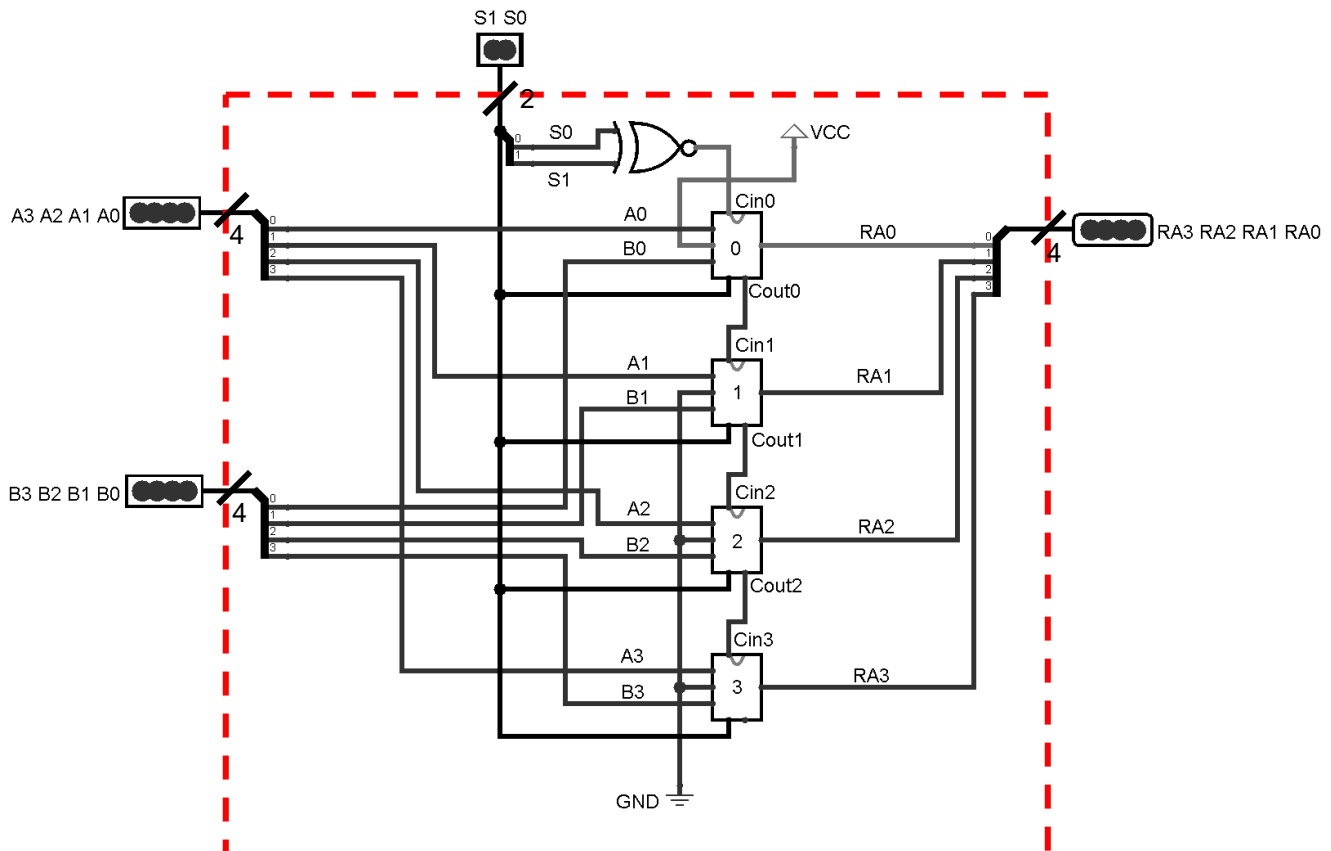
```

Nos falta por detallar el nivel 2 aritmético. Se consigue mediante la replicación 4 veces del nivel 1 aritmético, con la conexión de los acarrees de salida con los de entrada del peso siguiente, es decir,  $C_{out_i} = C_{in_{i+1}}$ .

El acarreo de salida final  $C_{out_3}$  es ignorado, como es habitual en las operaciones aritméticas en complemento a 2.

Respecto del acarreo de entrada inicial  $C_{in_0}$ , puede verse que coincide con el resultado de la operación XNOR con los bits  $S_1$  y  $S_0$ , ya que esta puerta devuelve 1 si ambos bits son iguales y 0 en caso contrario, justo lo que necesitamos.

Para terminar, a la miniALU de peso 0 le proporcionamos un bit  $D$  igual a 1. Para las otras tres miniALUs, este bit es 0. De esta forma, el valor completo de  $D$  con 4 bits es 0001.



```

(bit ra[4]) ALU_aritmetica_nivel2(bit a[4], bit b[4], bit s[2])
{
    bit ra[4]
    bit cout[4]
    (cout[0], ra[0]) = ALU_aritmetica_nivel1(a[0], b[0], s[0] XNOR s[1], 1, s)
    for (int i = 1; i<4; i++)
    {
        (cout[i], ra[i]) = ALU_aritmetica_nivel1(a[i], b[i], cout[i-1], 0, s)
    }
    return (ra)
}

```

6. **[Ejercicio de examen]** Se desea diseñar razonadamente y dibujar una unidad aritmético-lógica (ALU) para datos de 4 bits expresados en complemento a 2 ( $A_3A_2A_1A_0$  y  $B_3B_2B_1B_0$ ) que realice las siguientes operaciones:

| $S_1$ | $S_0$ | Operación                                                                                                                          |
|-------|-------|------------------------------------------------------------------------------------------------------------------------------------|
| 0     | 0     | $A + B$ (suma aritmética)                                                                                                          |
| 0     | 1     | $A - B$ (resta aritmética)                                                                                                         |
| 1     | 0     | Activar si $A < B$<br>Si se cumple la desigualdad, el resultado es 0001;<br>en caso contrario, 0000                                |
| 1     | 1     | $A == B$<br>Igualdad bit a bit<br>(si dos bits son iguales, el resultado de ese bit es 1;<br>si son diferentes, el resultado es 0) |

Para realizar este ejercicio se deberán tener en cuenta las siguientes consideraciones:

- El diseño debe ser modular, con lo que habrá que diseñar primero una ALU básica para operandos de 1 bit y particularizarla según la posición que ocupe dicho bit en los operandos  $A$  y  $B$ .
- Se podrán utilizar puertas lógicas auxiliares del tamaño necesario.
- Se podrán utilizar bloques combinatoriales auxiliares del tamaño necesario, como decodificadores, multiplexores, sumadores completos, etc.
- Habrá que indicar el valor de las señales de control auxiliares.

Las salidas de la ALU son las siguientes:

- Resultado  $R = R_3R_2R_1R_0$ .
- Indicador de resultado nulo  $Z$  (vale 1 si  $R = 0000$ ; en caso contrario, 0).
- Bit de desbordamiento  $V$  (vale 1 si hay desbordamiento en la suma o en la resta; si no hay desbordamiento en estas operaciones, vale 0; para las otras dos operaciones, siempre es 0).

Se pide:

- c) Dibujar y explicar cómo sería una **miniALU para datos de 1 bit** que realizara las 4 operaciones posibles.
- d) Dibujar y explicar cómo habría que enlazar entre sí las 4 miniALUs para conseguir la **ALU completa de 4 bits**.

### Solución:

a) La miniALU de 1 bit debe ser capaz de realizar 4 operaciones y de escoger 1 resultado, con lo que usaremos un multiplexor 4 a 1, controlado por las señales de selección  $S_1$  y  $S_0$ .

Para la suma y la resta usaremos un sumador completo de 1 bit, que estará conectado a las entradas 0 y 1 del multiplexor 4 a 1. Además, recibirá un acarreo de entrada definido en el nivel superior. Como primer sumando, recibirá el bit  $A_i$ , mientras que como segundo sumando podrá recibir o bien  $B_i$ , o bien  $\bar{B}_i$ , ya que para la resta en complemento a 2  $A - B$  necesitamos sumar  $A + \bar{B} + 1$ . Precisamente este 1 lo pasaremos a través del acarreo de entrada del bit 0 ( $C_{in0}$ ). Para elegir el valor adecuado del segundo sumando, usaremos un multiplexor de 2 a 1. Luego explicaremos con qué señal de selección.

Para calcular la operación activar si  $A < B$ , realizaremos también la resta  $A - B$ . Si el resultado es negativo, es porque se cumple la desigualdad. Lo que haremos será tomar el bit de signo de la resta (calculado como resultado de la miniALU de peso 3) y lo introduciremos en la miniALU de peso 0. Para las miniALU de pesos 1, 2 y 3 introduciremos un 0 directamente (conexión a tierra). A esta entrada de nuestra miniALU la llamaremos  $Menor_i$ , mientras que a la salida con el bit de signo de la miniALU de peso 3 la llamaremos simplemente  $Signo$ . Por tanto:

$$Menor_0 = Signo$$

$$Menor_1 = Menor_2 = Menor_3 = 0$$

Esto nos obliga a diseñar dos versiones de la miniALU de 1 bit: la del bit 3 y la de los bits 2, 1 y 0.

Para terminar, la comparación de igualdad bit a bit la realizaremos con una puerta XNOR, ya que devuelve 1 si los bits son iguales (00 o 11), o 0 si son diferentes (01 o 10).

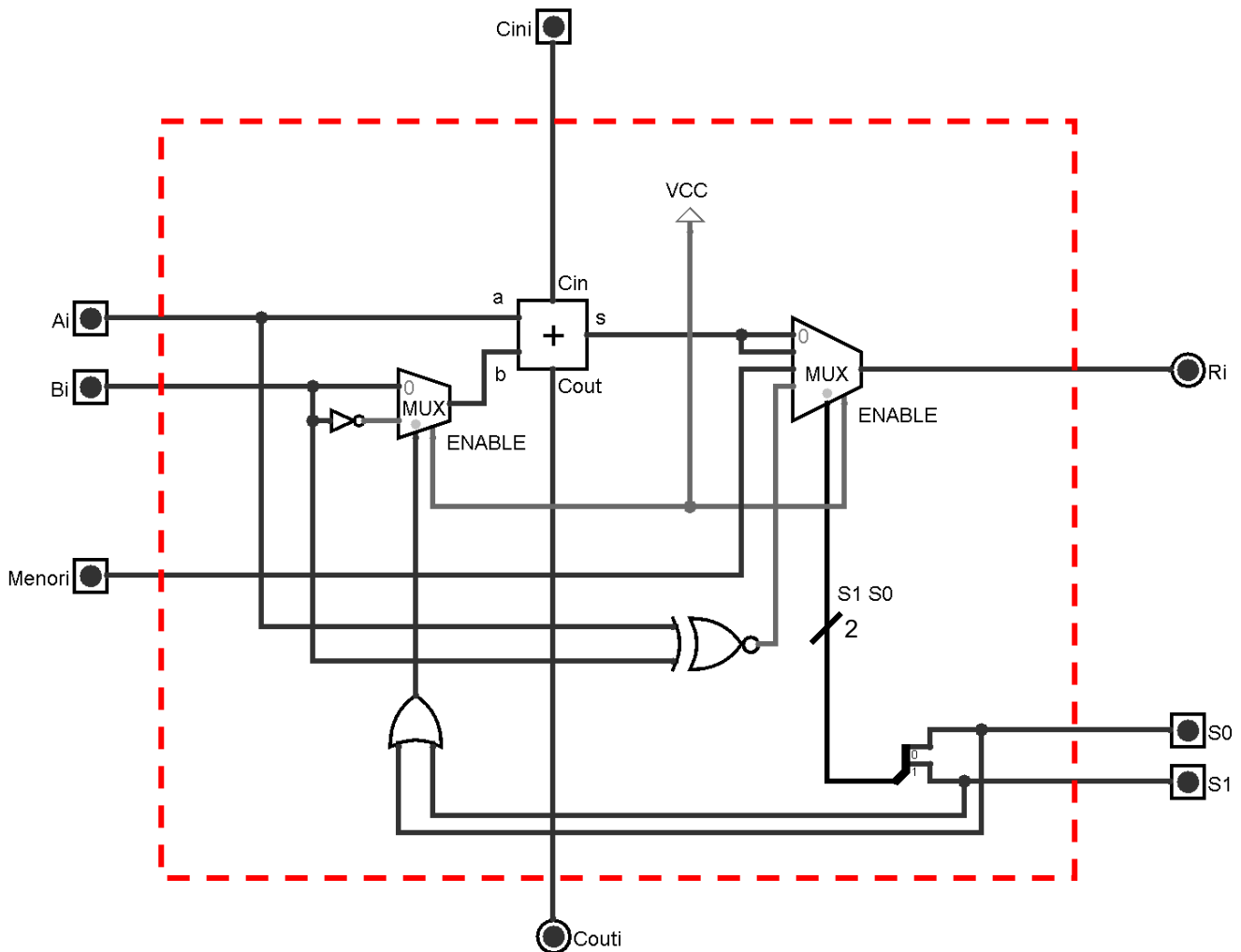
En cuanto al multiplexor de  $B$ , en la entrada 0 del mismo está  $B_i$  y en la 1 está  $\bar{B}_i$ . Nótese que para la resta (operación 01) y para activar si  $A < B$  (operación 10), necesitamos  $\bar{B}_i$ . Para la suma (operación 00) necesitamos  $B_i$ . Para la puerta XNOR, nos da igual el multiplexor, siempre y cuando conectemos con  $B_i$  directamente sin usar por el multiplexor. Por ello, la señal de selección sería:

| $S_1$ | $S_0$ | Necesitamos  | Señal de selección |
|-------|-------|--------------|--------------------|
| 0     | 0     | $B_i$        | 0                  |
| 0     | 1     | $\bar{B}_i$  | 1                  |
| 1     | 0     | $\bar{B}_i$  | 1                  |
| 1     | 1     | Nos da igual | X                  |

Un sencillo mapa de Karnaugh permite ver que la señal de selección que necesitamos puede calcularse como  $S_1 + S_0$ .

| $s_1 \backslash s_0$ | 0 | 1 |
|----------------------|---|---|
| 0                    | 0 | 1 |
| 1                    | 1 | X |

En la siguiente figura se muestra la estructura de la miniALU para un bit genérico  $i$  ( $0 \leq i \leq 2$ ):



```

(bit Couti, bit ri) ALU_1_bit_bits0a2(bit ai, bit bi, bit cini, bit menor_i,
bit s[2])
{
    bit enable = 1
    bit Couti, ri
    bit b, s
    bit entradasMux2a1_B[2]
    entradasMux2a1_B[0] = bi
    entradasMux2a1_B[1] = NOT bi

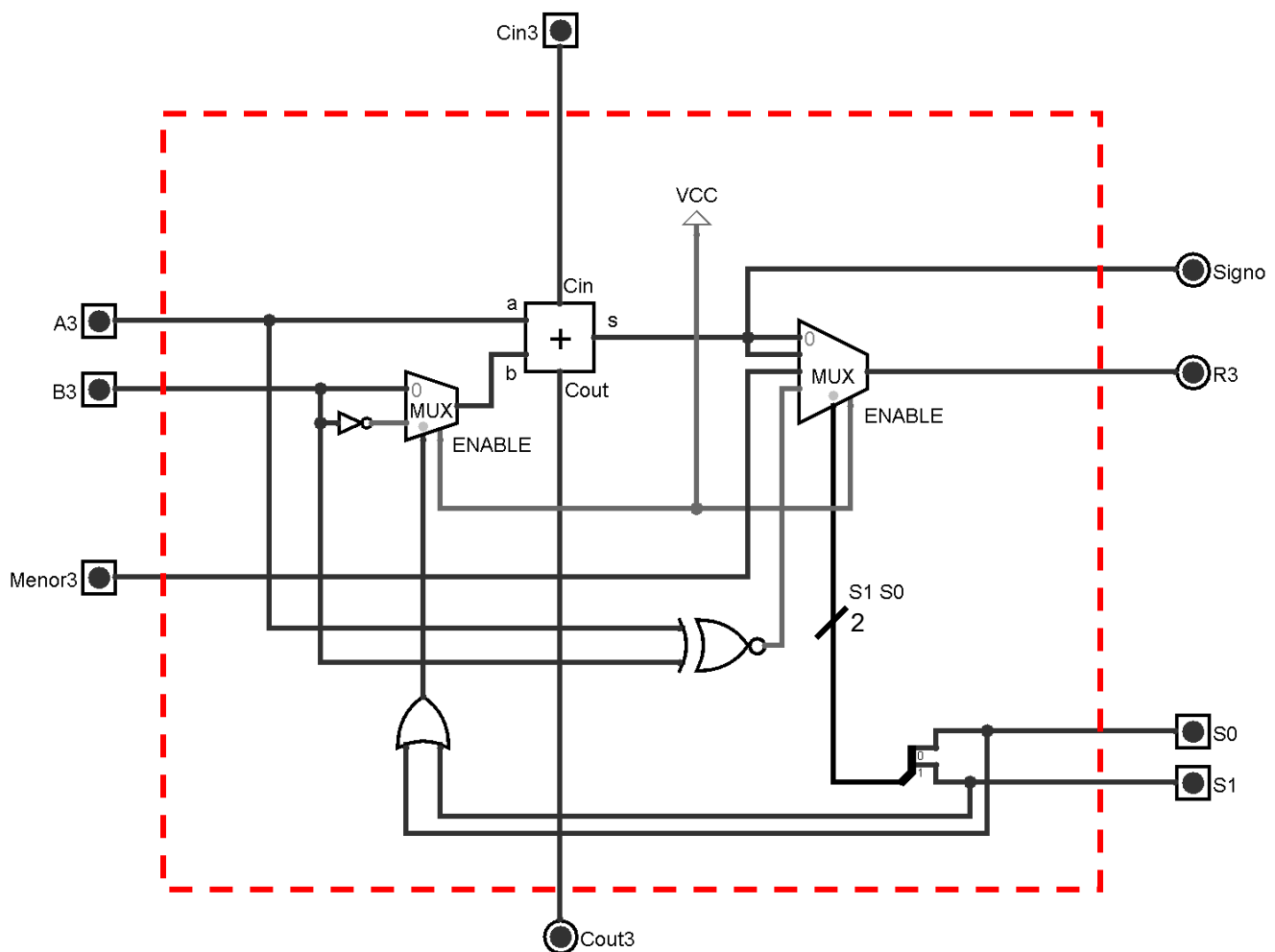
```

```

    b = mux2a1(entradasMux2a1_B, s[1] OR s[0], enable)
    (Couti, s) = sumadorCompleto_1_bit(ai, b, Cini)
    bit entradasMux4a1_R[4]
    entradasMux4a1_R[0] = s
    entradasMux4a1_R[1] = s
    entradasMux4a1_R[2] = menori
    entradasMux4a1_R[3] = ai XNOR bi
    ri = mux4a1(entradasMux4a1_R, s, enable)
    return (Couti, ri)
}

```

El bit  $i = 3$  sería ligeramente diferente para incluir la salida *Signo*:



```

(bit Cout3, bit r3, bit signo) ALU_1_bit_bit3(bit a3, bit b3, bit Cin3,
bit menor3, bit s[2])
{
    bit enable = 1

```

```

bit Cout3, r3
bit b, s
bit entradasMux2a1_B[2]
entradasMux2a1_B[0] = b3
entradasMux2a1_B[1] = NOT b3
b = mux2a1(entradasMux2a1_B, s[1] OR s[0], enable)
(Cout3, s) = sumadorCompleto_1_bit(a3, b, Cin3)
signo = s
bit entradasMux4a1_R[4]
entradasMux4a1_R[0] = s
entradasMux4a1_R[1] = s
entradasMux4a1_R[2] = menor3
entradasMux4a1_R[3] = a3 XNOR b3
r3 = mux4a1(entradasMux4a1_R, s, enable)
return (Cout3, r3, signo)
}

```

**b)** Para la ALU completa enlazaremos las 4 miniALUs anteriores (la de peso 3 y la otra repetida 3 veces para los pesos 2, 1 y 0, respectivamente). La salida *Signo* de la miniALU de peso 3 se conecta a la entrada *Menor*<sub>0</sub> de la miniALU de peso 0. Para las otras 3 miniALUs, la entrada *Menor* correspondiente está conectada a tierra (*GND*). De esta forma, el resultado de activar si  $A < B$  es 0000 cuando no se cumple la desigualdad (es decir, cuando la resta  $A - B$  da positivo o cero, con lo que el bit de signo es 0), o bien 0001 cuando sí se cumple (la resta es negativa y el bit de signo es 1).

Por otra parte, para el acarreo de entrada de la miniALU de peso 0 ( $C_{in0}$ ), debemos colocar unas puertas para que valga 0 en la suma y 1 en las operaciones de resta y de activar si  $A < B$  (para la operación de igualdad nos da igual porque se ignora este valor).

| $S_1$ | $S_0$ | $C_{in0}$ |
|-------|-------|-----------|
| 0     | 0     | 0         |
| 0     | 1     | 1         |
| 1     | 0     | 1         |
| 1     | 1     | X         |

Por un razonamiento similar a otro realizado anteriormente, puede verse que  $C_{in0} = S_1 + S_0$ .

Nótese que no podríamos haber conectado en las miniALUs la entrada  $C_{in}$  a la señal de selección del multiplexor de *B*, a pesar, ya que solo coinciden ambas señales para la miniALU de peso 0. Para las otras, el valor de  $C_{in}$  procede del  $C_{out}$  de la miniALU anterior.

En cuanto a los indicadores de resultado:

- Para indicar el resultado nulo *Z*, simplemente tenemos que usar una puerta NOR de 4 entradas:

$$Z = \overline{R_0 + R_1 + R_2 + R_3}$$

Ya que la combinación  $\overline{0 + 0 + 0 + 0}$  es la única que devuelve 1.

- En cuanto al bit indicador de desbordamiento  $V$ , hay que recordar que **NO** coincide con el acarreo  $C_{out_3}$  porque estamos trabajando en complemento a 2 (no en binario puro). Una manera sencilla de calcular este desbordamiento es comparando los dos últimos acarreos de salida:

$$V_{C2} = C_{out_3} \oplus C_{out_2} = C_{out_3} \oplus C_{in_3}$$

Y es así como lo hemos implementado, según se muestra en el dibujo siguiente. Otra manera equivalente es detectando un bit de signo del resultado incoherente con el de los operandos, es decir, es el mismo para los operandos, pero a la vez es distinto para el resultado:

$$V_{C2} = (\overline{A_3 \oplus B_3}) \cdot (A_3 \oplus R_3)$$

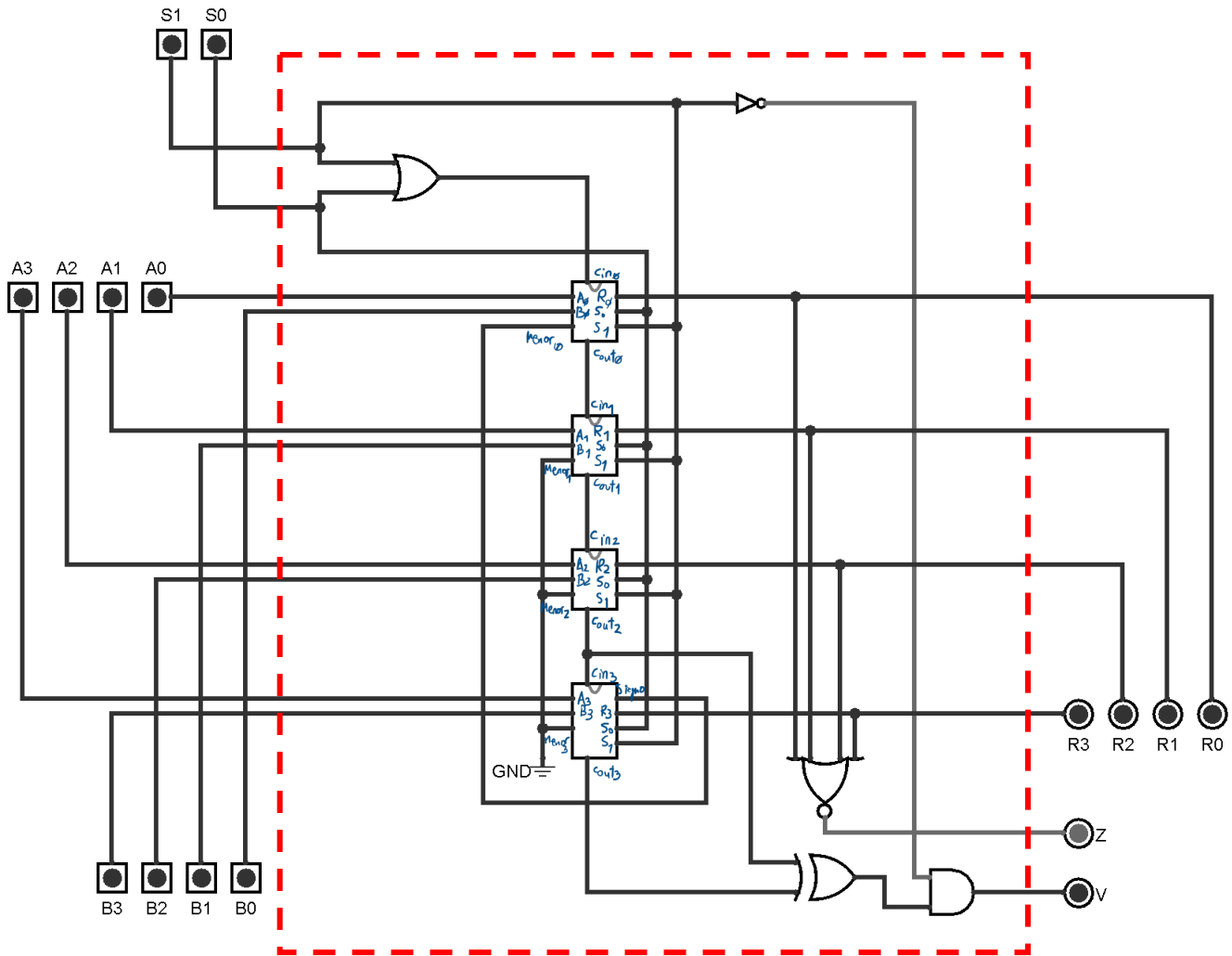
El enunciado dice también que este desbordamiento solo es válido para las operaciones de suma y de resta (cuando  $S_1 = 0$ ), mientras que debe ser obligatoriamente 0 para las otras dos operaciones. Por tanto, la versión definitiva del bit de desbordamiento es:

$$V = \bar{S}_1 \cdot V_{C2} = \bar{S}_1 (C_{out_3} \oplus C_{in_3})$$

NOTA: Por una errata del enunciado, también debería ser este el desbordamiento de la operación activar si  $A < B$ , ya que realiza internamente una resta en complemento a 2. Comprobar, por ejemplo, que en la comparación  $-5 < 5$  no se produce el resultado correcto (0001) porque hay desbordamiento ( $V_{C2} = 1$ , pero  $V = 0$  tal y como lo tenemos definido). Para que el desbordamiento fuera estrictamente correcto, debería ser:

$$V = (\overline{S_1 \cdot S_0}) (C_{out_3} \oplus C_{in_3})$$

Con todo esto en mente, el circuito de la ALU completa sería:



```

(bit r[4], bit z, bit v) ALU_4_bits(bit a[4], bit b[4], bit s[2])
{
    bit r[4]
    bit Cout[4]
    bit z, v
    (Cout[0], r[0]) = ALU_1_bit_bits0a2(a[0], b[0], s[1] OR s[0], signo, s)
    // menor0 ≡ signo lo genera la miniALU de peso 3
    z = r[0]
    for (int i = 1; i<3; i++)
    {
        (Cout[i], r[i]) = ALU_1_bit_bits0a2(a[i], b[i], Cout[i-1], 0, s)
        z = z OR r[i]
    }
    (Cout[3], r[3], signo) ALU_1_bit_bit3(a[3], b[3], Cout[2], 0, s)
    z = z OR r[3]
    z = NOT z
    v = (NOT s[1]) AND (Cout[2] XOR Cout[3])
    return (r, z, v)
}

```

7. [Ejercicio de examen] Se desea diseñar razonadamente y dibujar una unidad aritmético-lógica (ALU) para datos de 4 bits expresados en complemento a 2 ( $A_3A_2A_1A_0$  y  $B_3B_2B_1B_0$ ) que realice las siguientes operaciones en función de la señal de control  $S = S_2S_1S_0$ :

| $S_2$ | $S_1$ | $S_0$ | Operación                                       |
|-------|-------|-------|-------------------------------------------------|
| 0     | 0     | 0     | $A + B$ (suma aritmética)                       |
| 0     | 0     | 1     | $A - B$ (resta aritmética)                      |
| 0     | 1     | 0     | $A \cdot B$ (producto lógico)                   |
| 0     | 1     | 1     | $A + B$ (suma lógica)                           |
| 1     | 1     | 0     | $\overline{A \cdot B}$ (producto lógico negado) |
| 1     | 1     | 1     | Invierte los bits de A: $A_0A_1A_2A_3$          |

Recordatorio:  $\overline{A \cdot B} = \bar{A} + \bar{B}$

Para realizar este ejercicio se deberán tener en cuenta las siguientes consideraciones:

- El diseño debe ser modular, con lo que habrá que diseñar primero una ALU básica para operandos de 1 bit y particularizarla, si es necesario, según la posición que ocupe dicho bit en los operandos  $A$  y  $B$ .
- Se podrán utilizar puertas lógicas, así como sumadores completos de 1 bit y multiplexores, a modo de caja negra, es decir, sin detallar su estructura interna.
- Se dará prioridad a reutilizar componentes para que puedan realizar más de una operación.
- Habrá que calcular razonadamente el valor de las señales de control de los multiplexores (tabla de verdad y mapa de Karnaugh).
- Para las combinaciones de  $S$  no mencionadas en la tabla, el valor del resultado será 0000.

La única salida de la ALU es la siguiente:

- Resultado  $R = R_3R_2R_1R_0$ .

Se pide:

- Dibujar y explicar cómo sería una **miniALU para datos de 1 bit** que realizara las 6 operaciones posibles. Habrá que indicar y nombrar claramente cuáles son todas y cada una de las entradas y salidas de esta miniALU y escribir el **pseudocódigo** correspondiente.
- Dibujar y explicar cómo habría que enlazar entre sí las 4 miniALUs para conseguir la **ALU completa de 4 bits** y escribir el **pseudocódigo** correspondiente.

### Solución:

- Primero crearemos una miniALU para datos de 1 bit capaz de calcular todas las operaciones.

Aunque hay 6 operaciones, necesitaremos un multiplexor de 8 a 1, controlado por la señal  $S$ . Dos de sus entradas estarán conectadas a tierra, ya que se nos dice que el resultado en esos casos debe ser 0.

En cuanto al reaprovechamiento de componentes, usaremos un sumador completo para las operaciones 000 y 001. Asimismo, usaremos una puerta OR para las operaciones 011 y 110. Necesitaremos añadir dos multiplexores 2 a 1 para seleccionar los valores adecuados de las entradas de estos componentes.

Pensemos primero el multiplexor de A. En la entrada 0 pondremos el bit  $A_i$ , mientras que en la entrada 1 pondremos  $\bar{A}_i$ . La siguiente tabla muestra qué valores necesitamos para controlar este multiplexor.

| $S_2$ | $S_1$ | $S_0$ | Operación                                       | Valor necesario                                                    | Señal de selección del multiplexor de A |
|-------|-------|-------|-------------------------------------------------|--------------------------------------------------------------------|-----------------------------------------|
| 0     | 0     | 0     | $A + B$ (suma aritmética)                       | Conectaremos directamente con $A_i$<br>ya que no hace falta elegir | X                                       |
| 0     | 0     | 1     | $A - B$ (resta aritmética)                      | Conectaremos directamente con $A_i$<br>ya que no hace falta elegir | X                                       |
| 0     | 1     | 0     | $A \cdot B$ (producto lógico)                   | Conectaremos directamente con $A_i$<br>ya que no hace falta elegir | X                                       |
| 0     | 1     | 1     | $A + B$ (suma lógica)                           | $A_i$                                                              | 0                                       |
| 1     | 0     | 0     | Sin operación                                   | No interviene                                                      | X                                       |
| 1     | 0     | 1     | Sin operación                                   | No interviene                                                      | X                                       |
| 1     | 1     | 0     | $\overline{A \cdot B}$ (producto lógico negado) | $\bar{A}_i$                                                        | 1                                       |
| 1     | 1     | 1     | Invierte los bits de A:<br>$A_0A_1A_2A_3$       | No interviene                                                      | X                                       |

El mapa de Karnaugh correspondiente es:

| $S_2 \backslash S_1S_0$ | 00 | 01 | 11 | 10 |
|-------------------------|----|----|----|----|
| 0                       | X  | X  | 0  | X  |
| 1                       | X  | X  | X  | 1  |

Luego esta señal vale  $S_2$ . Otra posible simplificación equivalente habría sido  $\bar{S}_0$ . Para ahorrarnos la puerta NOT, preferimos elegir  $S_2$ .

En cuanto al multiplexor para  $B_i$ . En la entrada 0 pondremos el bit  $B_i$ , mientras que en la entrada 1 pondremos  $\bar{B}_i$ . La siguiente tabla muestra qué valores necesitamos para controlar este multiplexor.

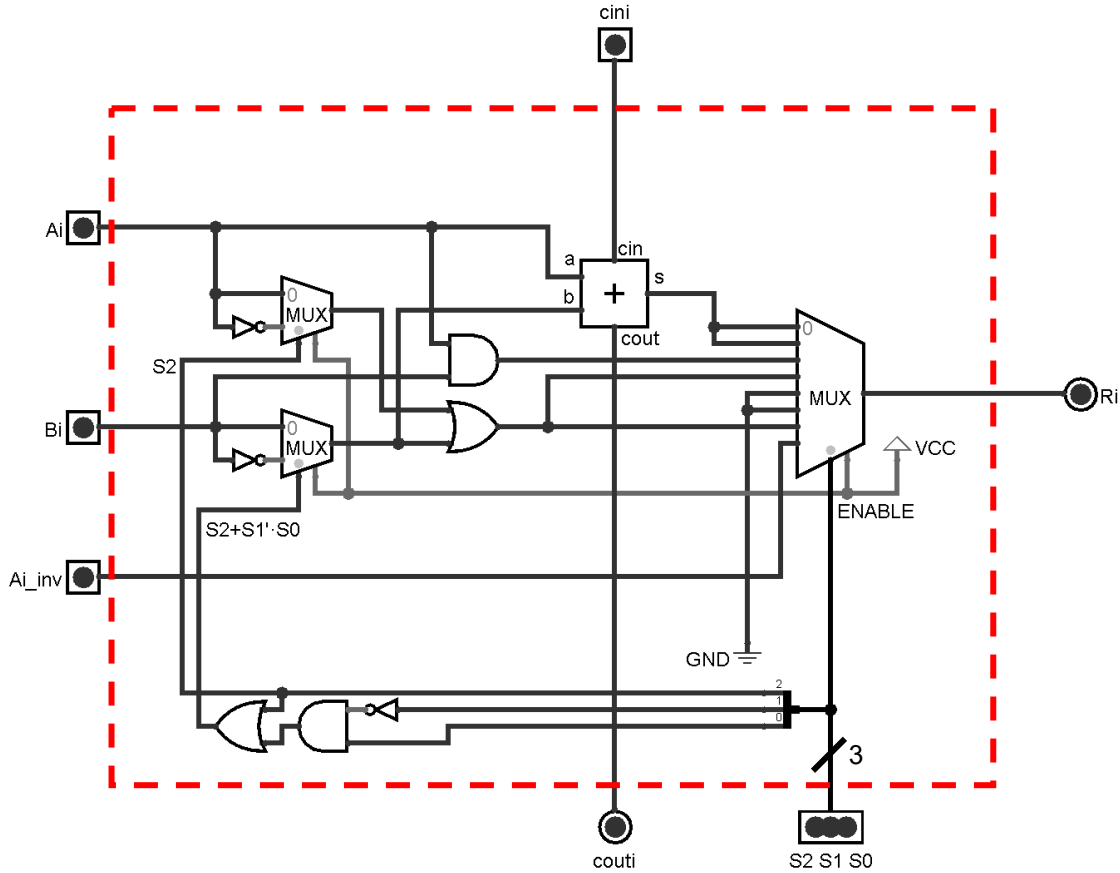
| $S_2$ | $S_1$ | $S_0$ | Operación                                       | Valor necesario                                                 | Señal de selección del multiplexor de B |
|-------|-------|-------|-------------------------------------------------|-----------------------------------------------------------------|-----------------------------------------|
| 0     | 0     | 0     | $A + B$ (suma aritmética)                       | $B_i$                                                           | 0                                       |
| 0     | 0     | 1     | $A - B$ (resta aritmética)                      | $\bar{B}_i$                                                     | 1                                       |
| 0     | 1     | 0     | $A \cdot B$ (producto lógico)                   | Conectaremos directamente con $B_i$ ya que no hace falta elegir | X                                       |
| 0     | 1     | 1     | $A + B$ (suma lógica)                           | $B_i$                                                           | 0                                       |
| 1     | 0     | 0     | Sin operación                                   | No interviene                                                   | X                                       |
| 1     | 0     | 1     | Sin operación                                   | No interviene                                                   | X                                       |
| 1     | 1     | 0     | $\overline{A \cdot B}$ (producto lógico negado) | $\bar{B}_i$                                                     | 1                                       |
| 1     | 1     | 1     | Invierte los bits de A:<br>$A_0A_1A_2A_3$       | No interviene                                                   | X                                       |

El mapa de Karnaugh correspondiente es:

| $S_2 \backslash S_1S_0$ | 00 | 01 | 11 | 10 |
|-------------------------|----|----|----|----|
| 0                       | 0  | 1  | 0  | X  |
| 1                       | X  | X  | X  | 1  |

Luego esta señal vale  $S_2 + \bar{S}_1S_0$ .

El circuito de la miniALU sería:



El código correspondiente en el pseudolenguaje de descripción de hardware sería:

```
(bit Couti, bit Ri) ALU_1_bit(bit Ai, bit Bi, bit Cini, bit Ai_inv, bit S[3])
{
    bit enable = 1
    bit entradasMux2a1_A[2]
    entradasMux2a1_A[0] = Ai
    entradasMux2a1_A[1] = NOT Ai
    bit A2 = mux2a1(entradasMux2a1_A, S[2], enable)
    bit entradasMux2a1_B[2]
    entradasMux2a1_B[0] = Bi
    entradasMux2a1_B[1] = NOT Bi
    bit B2 = mux2a1(entradasMux2a1_B, S[2] OR (NOT S[1] AND S[0]), enable)
    bit Couti, S, Ri
    (Couti, S) = sumadorCompleto_1_bit(Ai, B2, Cini)
    bit entradasMux8a1_R[8]
    entradasMux8a1_R[0] = S
    entradasMux8a1_R[1] = S
    entradasMux8a1_R[2] = Ai AND Bi
    entradasMux8a1_R[3] = A2 OR B2
    entradasMux8a1_R[4] = 0
    entradasMux8a1_R[5] = 0
}
```

```

    entradasMux8a1_R[6] = A2 OR B2
    entradasMux8a1_R[7] = Ai_inv
    Ri = mux8a1(entradasMux8a1_R, S, enable)
    return (Couti, Ri)
}

```

**NOTA:** Hemos usado un multiplexor 8 a 1 porque necesitamos seleccionar 5 posibles resultados: el del sumador completo, el de la puerta AND, el de la puerta OR, el bit cambiado de orden y el valor 0 para las operaciones no soportadas. Como puede verse, la señal de *enable* es fija y vale siempre 1 al estar conectada a *VCC*. Habría una manera de hacer este ejercicio con un multiplexor 4 a 1, pero en este caso la señal de *enable* hay que generarla a partir de los bits de *S*, lo cual implica escribir una tabla de verdad y realizar un mapa de Karnaugh. En este caso, la tabla de verdad de *enable* vale siempre 1 para las operaciones permitidas y 0 para las combinaciones 100 y 101 de *S*. De manera adicional, este multiplexor 4 a 1 tendría una señal de selección de 2 bits, que habría que calcular a partir de los 3 bits de *S*. Esto implica realizar otros dos mapas de Karnaugh adicionales. Se deja como ejercicio seguir esta estrategia.

En cuanto a la ALU completa, calculemos ahora el valor del acarreo de entrada inicial (solo necesario para las operaciones calculadas por el sumador completo):

| S <sub>2</sub> | S <sub>1</sub> | S <sub>0</sub> | Operación                                       | C <sub>in0</sub> |
|----------------|----------------|----------------|-------------------------------------------------|------------------|
| 0              | 0              | 0              | $A + B$ (suma aritmética)                       | 0                |
| 0              | 0              | 1              | $A - B$ (resta aritmética)                      | 1                |
| 0              | 1              | 0              | $A \cdot B$ (producto lógico)                   | X                |
| 0              | 1              | 1              | $A + B$ (suma lógica)                           | X                |
| 1              | 0              | 0              | Sin operación                                   | X                |
| 1              | 0              | 1              | Sin operación                                   | X                |
| 1              | 1              | 0              | $\overline{A \cdot B}$ (producto lógico negado) | X                |
| 1              | 1              | 1              | Invierte los bits de A: $A_0A_1A_2A_3$          | X                |

El mapa de Karnaugh correspondiente es:

| $S_2 \backslash S_1 S_0$ |  | 00 | 01 | 11 | 10 |
|--------------------------|--|----|----|----|----|
|                          |  | 0  | 1  | X  | X  |
| 0                        |  | 0  | 1  | X  | X  |
| 1                        |  | X  | X  | X  | X  |

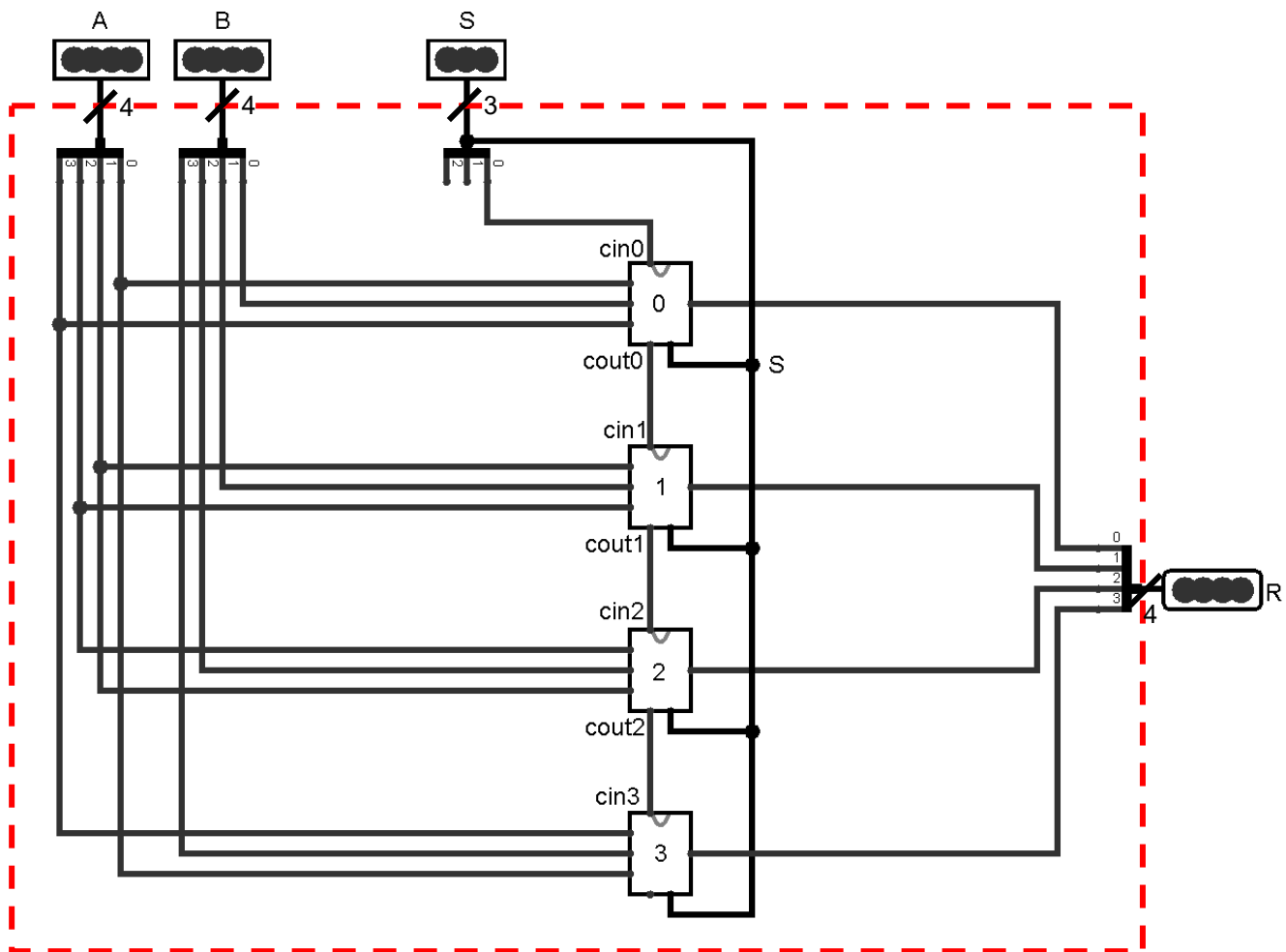
Luego esta señal vale  $S_0$ .

Por la entrada  $A_{i\_inv}$  de cada miniALU meteremos los siguientes valores:

| miniALU | Valor de la entrada $A_{i\_inv}$ |
|---------|----------------------------------|
| 0       | $A[3] = A[3 - 0]$                |
| 1       | $A[2] = A[3 - 1]$                |
| 2       | $A[1] = A[3 - 2]$                |
| 3       | $A[0] = A[3 - 3]$                |

Es decir, para la miniALU de peso  $i$ , se introduce el bit  $A[3 - i]$ .

La ALU completa sería:



El código correspondiente en el pseudolenguaje de descripción de hardware sería:

```
(bit R[4]) ALU_4_bits(bit A[4], bit B[4], bit S[3])
{
    bit R[4]
    bit Cout[4]
    (Cout[0], R[0]) = ALU_1_bit(A[0], B[0], S[0], A[3], S)
    for (int i = 1; i < 4; i++)
    {
        (Cout[i], R[i]) = ALU_1_bit(A[i], B[i], Cout[i-1], A[3-i], S)
    }
    return (R)
}
```

8. [Ejercicio de examen] Se desea diseñar razonadamente y dibujar una unidad aritmético-lógica (ALU) para datos de 4 bits expresados en complemento a 2 ( $A_3A_2A_1A_0$  y  $B_3B_2B_1B_0$ ) que realice las siguientes operaciones en función de la señal de control  $S = S_1S_0$ :

| $S_1$ | $S_0$ | Operación                  |
|-------|-------|----------------------------|
| 0     | 0     | $A + B$ (suma aritmética)  |
| 0     | 1     | $A \text{ AND } B$         |
| 1     | 0     | $A - B$ (resta aritmética) |
| 1     | 1     | $A \text{ NOR } B$         |

Recordatorio:  $\overline{A + B} = \bar{A} \cdot \bar{B}$

Para realizar este ejercicio se deberán tener en cuenta las siguientes consideraciones:

- El diseño debe ser modular, con lo que habrá que diseñar primero una ALU básica para operandos de 1 bit y particularizarla, si es necesario, según la posición que ocupe dicho bit en los operandos  $A$  y  $B$ .
- Se podrán utilizar puertas lógicas básicas, sumadores completos de 1 bit y multiplexores de cualquier tamaño, todos estos componentes utilizados a modo de caja negra, es decir, sin detallar su estructura interna.
- Se dará prioridad a reutilizar componentes para que puedan realizar más de una operación.
- Habrá que calcular razonadamente el valor de las señales de control de los multiplexores (tabla de verdad y mapa de Karnaugh, dibujando explícitamente los grupos).
- **No** será necesario escribir la notación de pseudocódigo correspondiente al circuito.

Las únicas salidas de la ALU son las siguientes:

- Resultado  $R = R_3R_2R_1R_0$ , calculado según la tabla anterior.
- Bit de desbordamiento aritmético  $V$  (se activa cuando corresponda según el resultado de la suma y la resta y vale siempre 0 para las otras dos operaciones).

Se pide:

- Dibujar y explicar cómo sería una **miniALU para datos de 1 bit** que realizara las 4 operaciones posibles. Habrá que indicar y nombrar claramente cuáles son todas y cada una de las entradas y salidas de esta miniALU.
- Dibujar y explicar cómo habría que enlazar entre sí las 4 miniALUs para conseguir la **ALU completa de 4 bits**.

**Solución:**

- La miniALU de 1 bit recibirá como entradas  $A_i$  y  $B_i$ , además del acarreo de entrada  $c_{in_i}$ . Las salidas serán el bit del resultado  $R_i$  y el acarreo de salida  $c_{out_i}$ .

Con afán de reutilizar componentes, la miniALU tendrá un único sumador completo, que permitirá hacer la suma y la resta, y una única puerta AND, que también servirá para la operación NOR, ya que  $A \text{ NOR } B = \overline{A + B} = \bar{A} \cdot \bar{B} = \bar{A} \text{ AND } \bar{B}$ . Para poder proporcionarles los operandos adecuados,

necesitaremos sendos multiplexores 2 a 1, que elegirán entre el valor original de un operando (entrada 0 del multiplexor) y el valor invertido (entrada 1).

De esta forma, como es habitual en este tipo de ejercicio, calcularemos todos los posibles resultados y lo seleccionaremos con un multiplexor. En principio, debería ser de tamaño 4 a 1, ya que son 4 las operaciones. La salida del sumador completo estaría conectada a las entradas 0 y 2 del multiplexor, mientras que la salida de la puerta AND estaría conectada a las entradas 1 y 3.

En este caso particular nos vale también un multiplexor de 2 a 1, ya que solo hay que elegir entre dos posibles resultados: el que da el sumador completo y el de la puerta AND. Esta será la opción escogida. La simplificación de la señal de control del multiplexor 2 a 1 es:

| $S_1 \backslash S_0$ | 0 | 1 |
|----------------------|---|---|
| 0                    | 0 | 1 |
| 1                    | 0 | 1 |

Luego esta señal es  $S_0$ .

La siguiente tabla indica los valores que necesitaremos para cada operación, el componente utilizado y la operación concreta que realiza, el valor de las entradas del componente ( $a$  y  $b$ ) y el valor de las señales de selección de ambos multiplexor, así como el valor del acarreo de entrada de la miniALU de peso 0. Nótese que el sumador completo siempre utiliza el valor  $A_i$ , con lo que conectaremos este valor directamente con la entrada  $a$  del sumador sin pasar por el multiplexor.

| $S_1$ | $S_0$ | Operación                  | Componente       | Operación en componente            | $a$         | Señal MUX $A_i$ | $b$         | Señal MUX $B_i$ | $c_{in0}$ |
|-------|-------|----------------------------|------------------|------------------------------------|-------------|-----------------|-------------|-----------------|-----------|
| 0     | 0     | $A + B$ (suma aritmética)  | Sumador completo | $A_i + B_i + c_{in_i}$             | $A_i$       | X               | $B_i$       | 0               | 0         |
| 0     | 1     | $A \text{ AND } B$         | Puerta AND       | $A_i \text{ AND } B_i$             | $A_i$       | 0               | $B_i$       | 0               | X         |
| 1     | 0     | $A - B$ (resta aritmética) | Sumador completo | $A_i + \bar{B}_i + c_{in_i}$       | $A_i$       | X               | $\bar{B}_i$ | 1               | 1         |
| 1     | 1     | $A \text{ NOR } B$         | Puerta AND       | $\bar{A}_i \text{ AND } \bar{B}_i$ | $\bar{A}_i$ | 1               | $\bar{B}_i$ | 1               | X         |

La simplificación de la señal de control del multiplexor de  $A_i$  es:

| $S_1 \backslash S_0$ | 0 | 1 |
|----------------------|---|---|
| 0                    | X | 0 |
| 1                    | X | 1 |

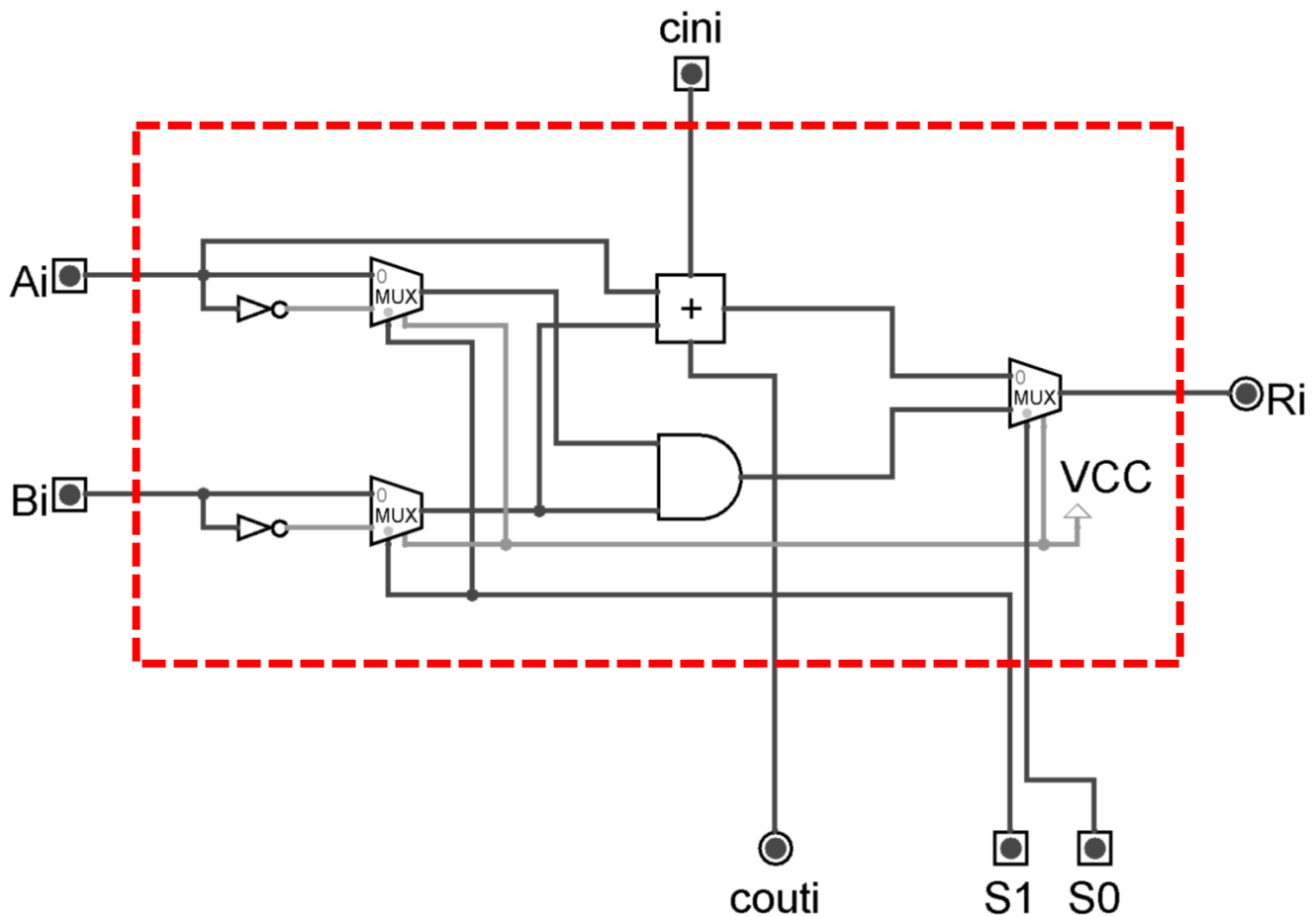
Luego esta señal vale  $S_1$ .

La simplificación de la señal de control del multiplexor de  $B_i$  es:

| $S_1 \backslash S_0$ | 0 | 1 |
|----------------------|---|---|
| 0                    | 0 | 0 |
| 1                    | 1 | 1 |

Luego esta señal también vale  $S_1$ .

El circuito de la miniALU de peso  $i$  ( $0 \leq i \leq 3$ ) será:



El valor del acarreo de entrada de la miniALU de peso 0,  $c_{in_0}$ , valdrá:

| $S_1 \backslash S_0$ | 0 | 1 |
|----------------------|---|---|
| 0                    | 0 | X |
| 1                    | 1 | X |

Luego esta señal también vale  $S_1$ .

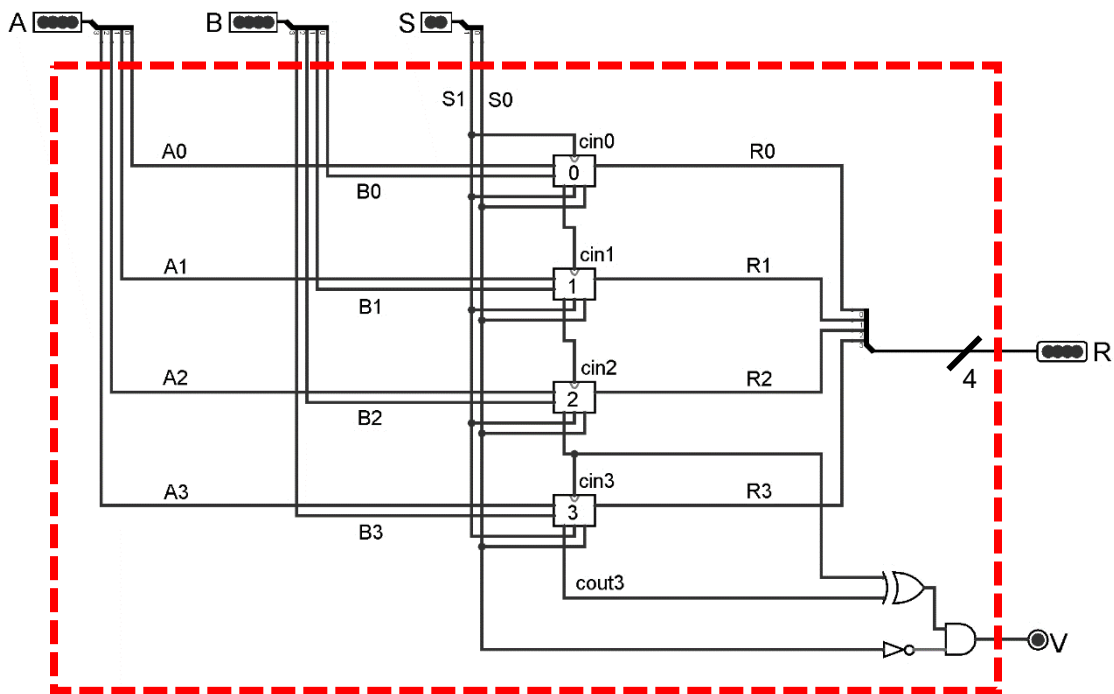
En cuanto al valor de la señal de desbordamiento  $V$ , su tabla de verdad es:

| $S_1$ | $S_0$ | Operación                  | $V$                         |
|-------|-------|----------------------------|-----------------------------|
| 0     | 0     | $A + B$ (suma aritmética)  | $c_{in_3} \oplus c_{out_3}$ |
| 0     | 1     | $A \text{ AND } B$         | 0                           |
| 1     | 0     | $A - B$ (resta aritmética) | $c_{in_3} \oplus c_{out_3}$ |
| 1     | 1     | $A \text{ NOR } B$         | 0                           |

Para garantizar que  $V$  se anula en las operaciones calculadas con la puerta AND, el valor definitivo de esta señal será:

$$V = (c_{in_3} \oplus c_{out_3}) \cdot \bar{S}_0$$

En cuanto al circuito total, se crearía por concatenación de 4 miniALUs de 1 bit, conectando  $c_{out_i}$  a  $c_{in_{i+1}}$  para  $0 \leq i \leq 2$ .



Bonus track: aunque el enunciado del examen no lo pedía, vamos a escribir también el código correspondiente en el pseudolenguaje de descripción de hardware:

```
(bit Couti, bit Ri) ALU_1_bit(bit Ai, bit Bi, bit Cini, bit S[2])
{
    bit enable = 1
    bit entradasMux2a1_A[2]
    entradasMux2a1_A[0] = Ai
    entradasMux2a1_A[1] = NOT Ai
    bit A2 = mux2a1(entradasMux2a1_A, S[1], enable)
    bit entradasMux2a1_B[2]
    entradasMux2a1_B[0] = Bi
    entradasMux2a1_B[1] = NOT Bi
    bit B2 = mux2a1(entradasMux2a1_B, S[1], enable)
    bit Couti, S, Ri
    (Couti, S) = sumadorCompleto_1_bit(Ai, B2, Cini)
    bit entradasMux2a1_R[2]
    entradasMux2a1_R[0] = S
    entradasMux2a1_R[1] = A2 AND B2
    Ri = mux8a1(entradasMux2a1_R, S[0], enable)
    return (Couti, Ri)
}
```

```
(bit R[4], bit V) ALU_4_bits(bit A[4], bit B[4], bit S[2])
{
    bit R[4]
    bit Cout[4]
    (Cout[0], R[0]) = ALU_1_bit(A[0], B[0], S[1], S)
    for (int i = 1; i < 4; i++)
    {
        (Cout[i], R[i]) = ALU_1_bit(A[i], B[i], Cout[i-1], S)
    }
    bit V = (Cout[3] XOR Cout[2]) AND (NOT S[0])
    return (R, V)
}
```

9. [Ejercicio de examen] Se desea diseñar razonadamente y dibujar una unidad aritmético-lógica (ALU) para datos de 4 bits expresados en complemento a 2 ( $A_3A_2A_1A_0$  y  $B_3B_2B_1B_0$ ) que realice las siguientes operaciones en función de la señal de control  $S = S_2S_1S_0$ :

| $S_2$ | $S_1$ | $S_0$ | Operación                                 |
|-------|-------|-------|-------------------------------------------|
| 0     | 0     | 0     | $A - B$ (resta aritmética)                |
| 0     | 0     | 1     | $A \cdot B$ (producto lógico)             |
| 0     | 1     | 0     | $A + B$ (suma lógica)                     |
| 0     | 1     | 1     | Operación prohibida                       |
| 1     | 0     | 0     | $\bar{A} + \bar{B}$ (suma lógica)         |
| 1     | 0     | 1     | $A + B$ (suma aritmética)                 |
| 1     | 1     | 0     | Operación prohibida                       |
| 1     | 1     | 1     | $\bar{A} \cdot \bar{B}$ (producto lógico) |

Para realizar este ejercicio se deberán tener en cuenta las siguientes consideraciones:

- El diseño debe ser modular, con lo que habrá que diseñar primero una ALU básica para operandos de 1 bit y particularizarla, si es necesario, según la posición que ocupe dicho bit en los operandos  $A$  y  $B$ .
- Se podrán utilizar puertas lógicas, así como sumadores completos de 1 bit y multiplexores del tamaño que se necesite. Los multiplexores deberán tener señal enable.
- Siempre que se use un componente a modo de caja negra, será obligatorio indicar los nombres de cada una de sus entradas y salidas.
- Se dará prioridad a reutilizar componentes para que puedan realizar más de una operación.
- Se dará prioridad a utilizar el menor número de componentes y del menor tamaño posible.
- Habrá que calcular razonadamente el valor de las señales de control de los multiplexores utilizados (tabla de verdad y mapa de Karnaugh).
- Para las combinaciones de  $S$  correspondientes a las operaciones prohibidas, la ALU devuelve cero, independientemente de los valores de  $A$  y  $B$ .

Las dos únicas salidas de la ALU son las siguientes:

- El resultado  $R = R_3R_2R_1R_0$ .
- Un bit  $Z$ , que se activa si y solo si  $R$  vale 0.

Se pide:

- Dibujar y explicar cómo sería una **miniALU para datos de 1 bit** que realizara todas las operaciones posibles. Habrá que indicar y nombrar claramente cuáles son todas y cada una de las entradas y salidas de esta miniALU.
- Dibujar y explicar cómo habría que enlazar entre sí las 4 miniALUs para conseguir la **ALU completa de 4 bits**.

## Solución:

a) Como en todo ejercicio de diseño ingenieril, puede haber más de una solución para un problema. En este caso, vamos a ver tres posibles soluciones.

### ALTERNATIVA 1:

Como hay 8 operaciones (incluyendo las 2 que siempre devuelven cero), usaremos un multiplexor 8 a 1 para elegir el resultado. La señal de selección de este multiplexor son directamente los 3 bits de  $S$ .

Usaremos una puerta AND en las operaciones 1 y 7, pero deberemos proporcionarle las entradas adecuadas ( $A_i$  o  $\bar{A}_i$ , por un lado, y  $B_i$  o  $\bar{B}_i$ , por otro).

La puerta OR la utilizaremos en la operación 2 y en la 4 y también le tendremos que proporcionar las entradas adecuadas ( $A_i$  o  $\bar{A}_i$ , por un lado, y  $B_i$  o  $\bar{B}_i$ , por otro).

Finalmente, en cuanto al sumador completo, lo usaremos en las operaciones 0 y 5. Para el operando  $a$  siempre nos hará falta  $A_i$ , con lo que conectaremos directamente esta entrada  $a$  con la entrada  $A_i$  de la miniALU. Sin embargo, como la resta  $A - B$  se convierte en  $A + \bar{B} + 1$ , para el operando  $b$ , necesitaremos  $B_i$  o  $\bar{B}_i$  según si estemos sumando o restando.

Por tanto, usaremos dos multiplexores 2 a 1: uno para elegir entre  $A_i$  (entrada 0) o  $\bar{A}_i$  (entrada 1), por un lado, y entre  $B_i$  (entrada 0) o  $\bar{B}_i$  (entrada 1), por otro.

A continuación, puede verse la tabla de verdad del multiplexor que elige entre  $A_i$  o  $\bar{A}_i$ :

| $S_2$ | $S_1$ | $S_0$ | Operación                                 | Valor necesario | Señal de selección |
|-------|-------|-------|-------------------------------------------|-----------------|--------------------|
| 0     | 0     | 0     | $A - B$ (resta aritmética)                | No interviene   | X                  |
| 0     | 0     | 1     | $A \cdot B$ (producto lógico)             | $A_i$           | 0                  |
| 0     | 1     | 0     | $A + B$ (suma lógica)                     | $A_i$           | 0                  |
| 0     | 1     | 1     | Operación prohibida                       | No interviene   | X                  |
| 1     | 0     | 0     | $\bar{A} + \bar{B}$ (suma lógica)         | $\bar{A}_i$     | 1                  |
| 1     | 0     | 1     | $A + B$ (suma aritmética)                 | No interviene   | X                  |
| 1     | 1     | 0     | Operación prohibida                       | No interviene   | X                  |
| 1     | 1     | 1     | $\bar{A} \cdot \bar{B}$ (producto lógico) | $\bar{A}_i$     | 1                  |

El mapa de Karnaugh para esta señal es:

| $S_2 \backslash S_1 S_0$ | 00 | 01 | 11 | 10 |
|--------------------------|----|----|----|----|
| 0                        | X  | 0  | X  | 0  |
| 1                        | 1  | X  | 1  | X  |

El valor de esta señal es  $S_2$ .

Hagamos lo mismo con la señal de selección del multiplexor que elige entre  $B_i$  o  $\bar{B}_i$ :

| $S_2$ | $S_1$ | $S_0$ | Operación                                 | Valor necesario | Señal de selección |
|-------|-------|-------|-------------------------------------------|-----------------|--------------------|
| 0     | 0     | 0     | $A - B$ (resta aritmética)                | $\bar{B}_i$     | 1                  |
| 0     | 0     | 1     | $A \cdot B$ (producto lógico)             | $B_i$           | 0                  |
| 0     | 1     | 0     | $A + B$ (suma lógica)                     | $B_i$           | 0                  |
| 0     | 1     | 1     | Operación prohibida                       | No interviene   | X                  |
| 1     | 0     | 0     | $\bar{A} + \bar{B}$ (suma lógica)         | $\bar{B}_i$     | 1                  |
| 1     | 0     | 1     | $A + B$ (suma aritmética)                 | $B_i$           | 0                  |
| 1     | 1     | 0     | Operación prohibida                       | No interviene   | X                  |
| 1     | 1     | 1     | $\bar{A} \cdot \bar{B}$ (producto lógico) | $\bar{B}_i$     | 1                  |

El mapa de Karnaugh para esta señal es:

| $S_2 \backslash S_1 S_0$ | 00 | 01 | 11 | 10 |
|--------------------------|----|----|----|----|
| 0                        | 1  | 0  | X  | 0  |
| 1                        | 1  | 0  | 1  | X  |

El valor de esta señal es  $\bar{S}_1 \bar{S}_0 + S_1 S_0$ .

Otro posible mapa de Karnaugh sería:

| $S_2 \backslash S_1 S_0$ | 00 | 01 | 11 | 10 |
|--------------------------|----|----|----|----|
| 0                        | 1  | 0  | X  | 0  |
| 1                        | 1  | 0  | 1  | X  |

El valor de esta señal es  $\bar{S}_1 \bar{S}_0 + S_2 S_1$ . Esta simplificación es equivalente a la anterior. En todo caso, solo hay 2 grupos de unos y, por tanto, 2 términos de producto sumados (tomar 3 grupos de unos sería redundante).

Podemos simplificar un poco el circuito anterior aplicando un truco que se cumple casualmente para este ejercicio. Se trata de aplicar las leyes de de Morgan en las operaciones lógicas:

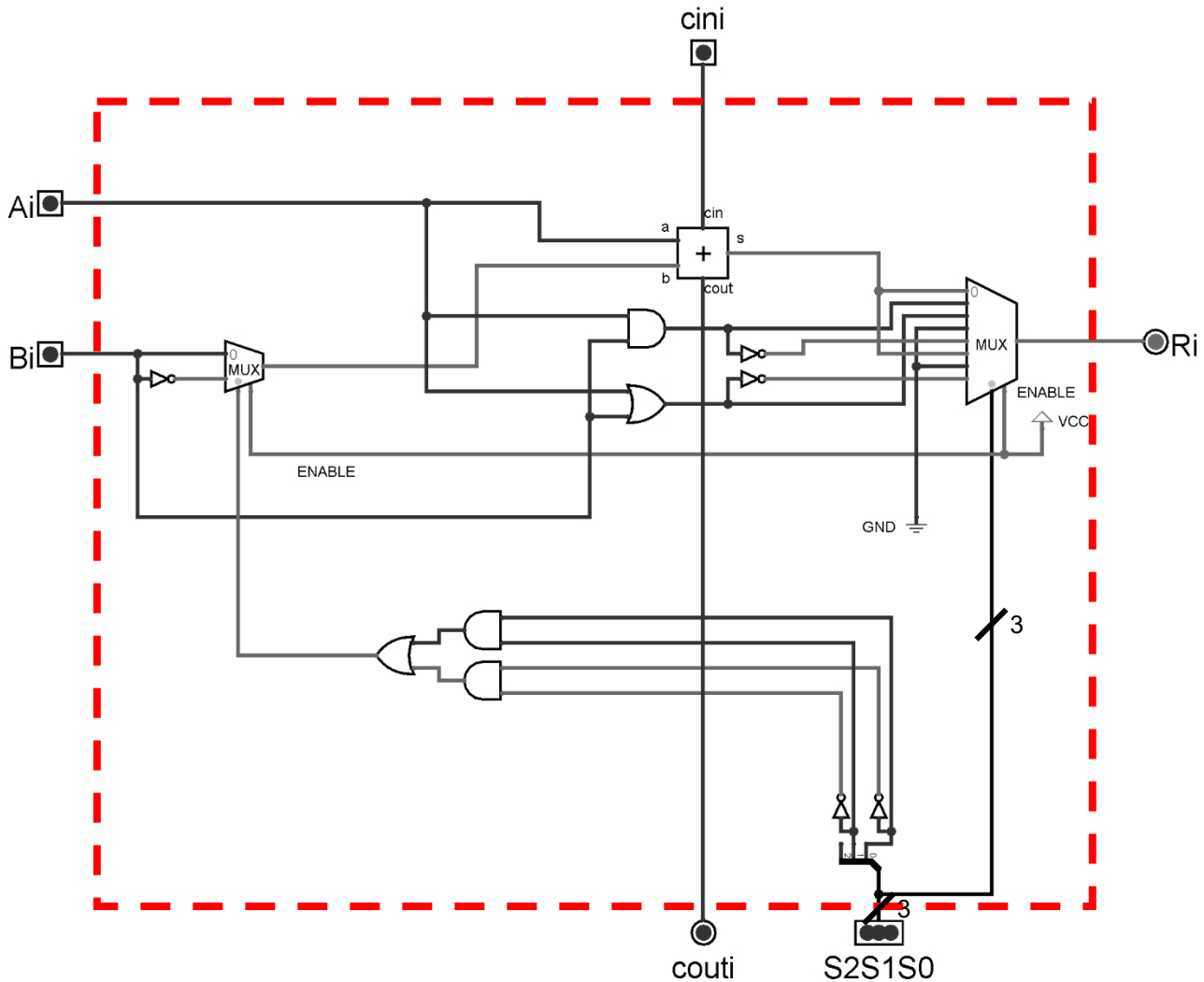
- 50 -

En esta alternativa, usaremos solo un multiplexor 2 a 1 para elegir el valor adecuado del segundo operando del sumador (entre  $B_i$  y  $\bar{B}_i$ ).

Colocaremos también unas puertas AND y OR, que recibirán como entradas directamente  $A_i$  o  $B_i$ . El resultado de cada puerta lo invertiremos con sendas puertas NOT.

La puerta AND estará conectada a la entrada 1 del multiplexor 8 a 1, mientras que la AND y la NOT lo hará a la entrada 4. La puerta OR estará conectada a la entrada 2, mientras que la OR y la NOT lo hará a la 7.

Por tanto, el circuito de la miniALU en su versión 2 sería el siguiente:



### **ALTERNATIVA 3:**

Podemos darnos cuenta de que en realidad solo necesitamos tres componentes para implementar este circuito: una puerta AND, una puerta OR y un sumador completo. Por tanto, vamos a utilizar un multiplexor 4 a 1 para elegir el resultado. Nos basaremos en la alternativa 1.

En cuanto al multiplexor 4 a 1 para el resultado, tendría una señal de selección de 2 bits. Supongamos que la salida  $s$  del sumador completo está conectada a la entrada 0, la puerta AND a la entrada 1, la puerta OR a la entrada 2 y la tierra a la entrada 3. Entonces esta sería la tabla de verdad:

| $S_2$ | $S_1$ | $S_0$ | Operación                                 | Señal de selección |
|-------|-------|-------|-------------------------------------------|--------------------|
| 0     | 0     | 0     | $A - B$ (resta aritmética)                | 00                 |
| 0     | 0     | 1     | $A \cdot B$ (producto lógico)             | 01                 |
| 0     | 1     | 0     | $A + B$ (suma lógica)                     | 10                 |
| 0     | 1     | 1     | Operación prohibida                       | 11                 |
| 1     | 0     | 0     | $\bar{A} + \bar{B}$ (suma lógica)         | 10                 |
| 1     | 0     | 1     | $A + B$ (suma aritmética)                 | 00                 |
| 1     | 1     | 0     | Operación prohibida                       | 11                 |
| 1     | 1     | 1     | $\bar{A} \cdot \bar{B}$ (producto lógico) | 01                 |

El mapa de Karnaugh del bit más significativo de esta señal sería:

| $S_2 \backslash S_1 S_0$ | 00 | 01 | 11 | 10 |
|--------------------------|----|----|----|----|
| 0                        | 0  | 0  | 1  | 1  |
| 1                        | 1  | 0  | 0  | 1  |

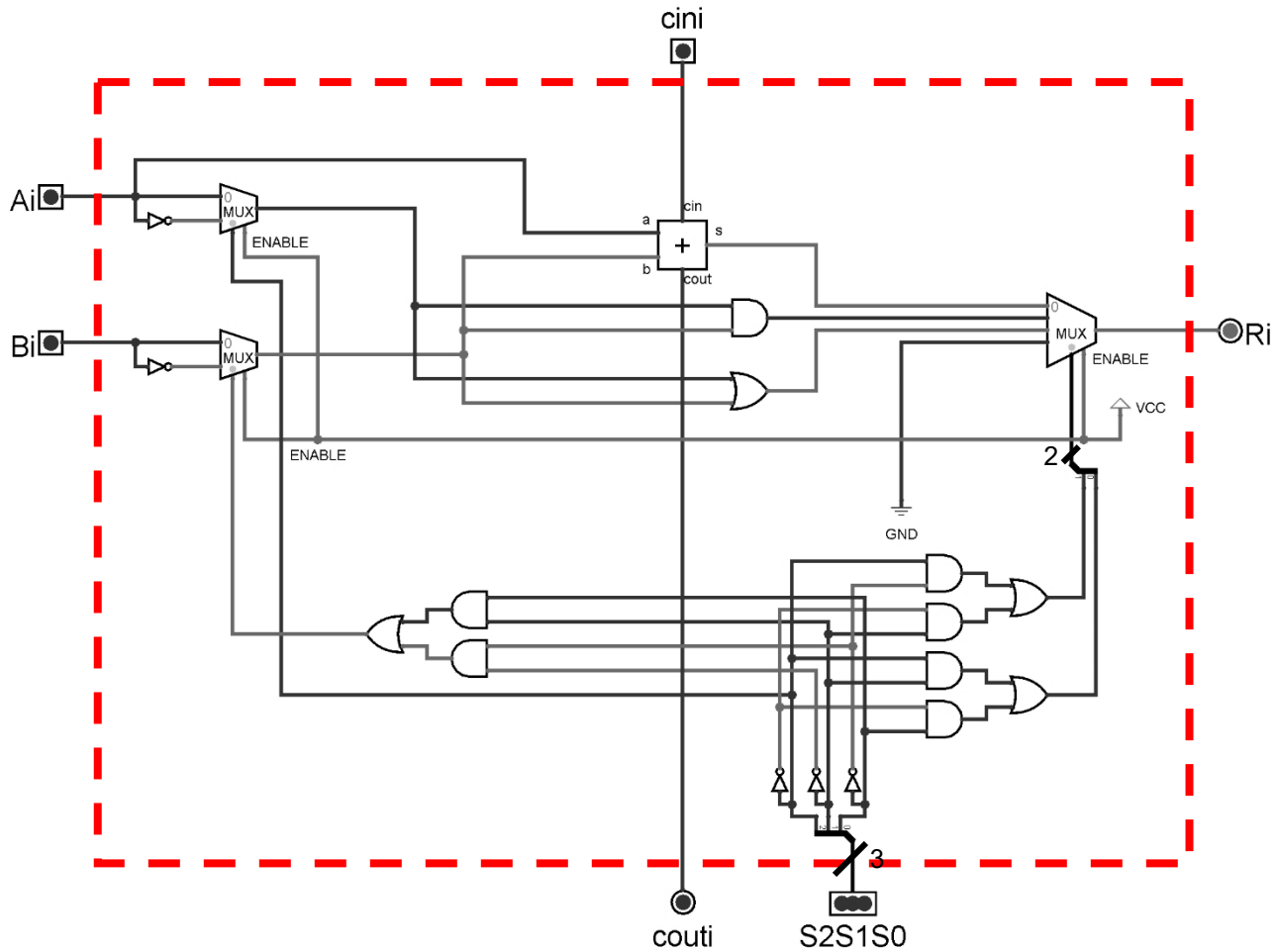
Este bit valdría  $S_2 \bar{S}_0 + \bar{S}_2 S_1$ .

El mapa de Karnaugh del bit menos significativo de esta señal sería:

| $S_2 \backslash S_1 S_0$ | 00 | 01 | 11 | 10 |
|--------------------------|----|----|----|----|
| 0                        | 0  | 1  | 1  | 0  |
| 1                        | 0  | 0  | 1  | 1  |

Este bit valdría  $S_2 S_1 + \bar{S}_2 S_0$ .

Por tanto, el circuito de la miniALU en su versión 3 sería el siguiente:



**b)** En cuanto a la ALU completa, bastará con enlazar las cuatro miniALUs para propagar los acarrees que necesitan las dos operaciones aritméticas. Con relación a esto, necesitamos calcular el acarreo de entrada  $c_{in0}$ .

| $S_2$ | $S_1$ | $S_0$ | Operación                                 | $c_{in0}$ |
|-------|-------|-------|-------------------------------------------|-----------|
| 0     | 0     | 0     | $A - B$ (resta aritmética)                | 1         |
| 0     | 0     | 1     | $A \cdot B$ (producto lógico)             | X         |
| 0     | 1     | 0     | $A + B$ (suma lógica)                     | X         |
| 0     | 1     | 1     | Operación prohibida                       | X         |
| 1     | 0     | 0     | $\bar{A} + \bar{B}$ (suma lógica)         | X         |
| 1     | 0     | 1     | $A + B$ (suma aritmética)                 | 0         |
| 1     | 1     | 0     | Operación prohibida                       | X         |
| 1     | 1     | 1     | $\bar{A} \cdot \bar{B}$ (producto lógico) | X         |

El mapa de Karnaugh de este acarreo es:

| $S_2 \backslash S_1 S_0$ | 00 | 01 | 11 | 10 |
|--------------------------|----|----|----|----|
| 0                        | 1  | X  | X  | X  |
| 1                        | X  | 0  | X  | X  |

Por tanto, el acarreo  $c_{in0}$  vale  $\bar{S}_2$ .

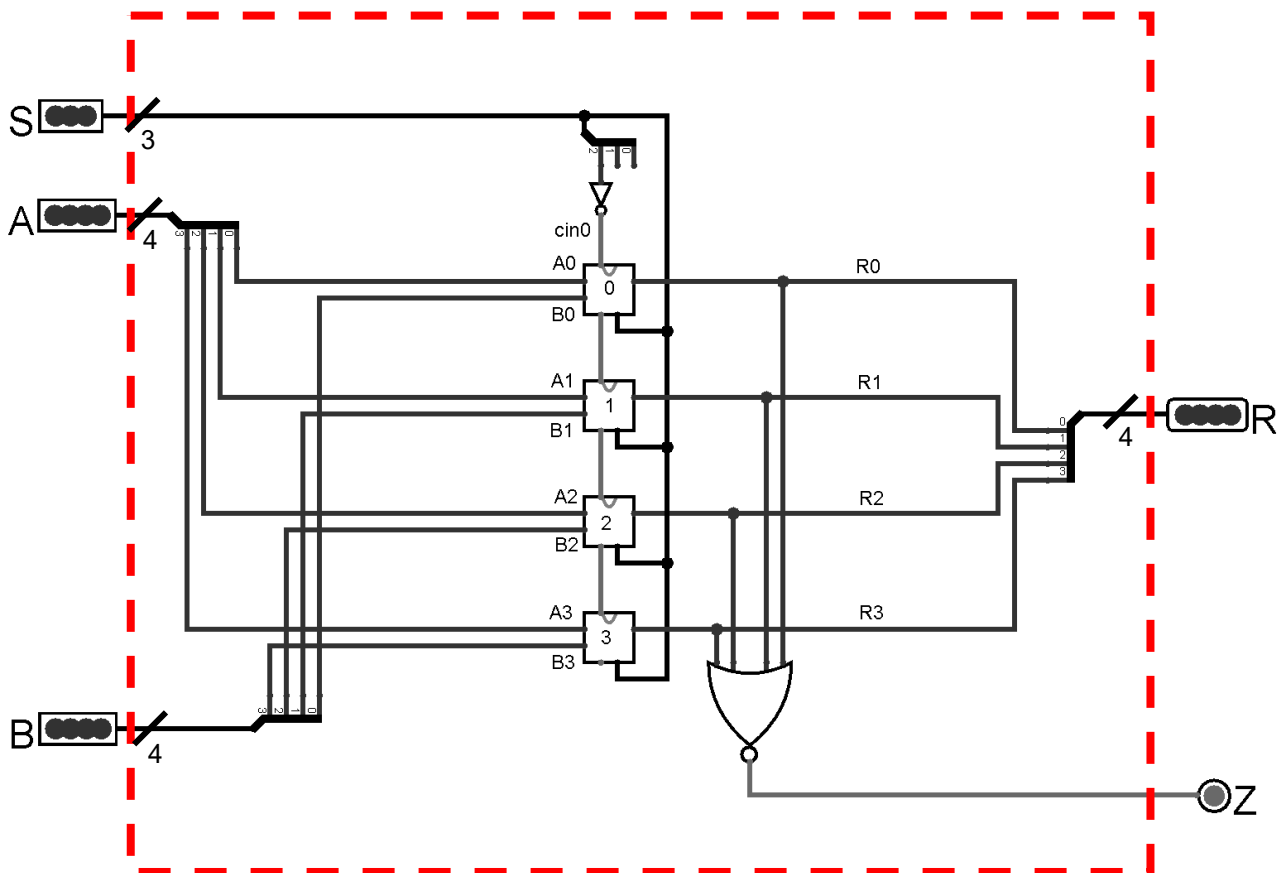
Otro posible mapa de Karnaugh sería:

| $S_2 \backslash S_1 S_0$ | 00 | 01 | 11 | 10 |
|--------------------------|----|----|----|----|
| 0                        | 1  | X  | X  | X  |
| 1                        | X  | 0  | X  | X  |

Según este mapa, el acarreo valdría  $\bar{S}_0$ . Esta solución es equivalente a la anterior (pero sería erróneo elegir los dos grupos a la vez porque son redundantes).

En cuanto al detector de ceros  $Z$ , usaremos una puerta NOR de 4 entradas conectada a cada uno de los bits del resultado.

El circuito de la ALU completa es:



Nótese que el enunciado no pedía escribir el pseudocódigo equivalente a estos circuitos. Se deja como ejercicio para practicar.

10. [Ejercicio de examen] Se desea diseñar **razonadamente** y dibujar una unidad aritmético-lógica (ALU) para datos de 4 bits expresados en complemento a 2 ( $A_3A_2A_1A_0$  y  $B_3B_2B_1B_0$ ) que realice las siguientes operaciones aritméticas en función de la señal de control  $S = S_1S_0$ :

| $S_1$ | $S_0$ | Operación   |
|-------|-------|-------------|
| 0     | 0     | $A + B$     |
| 0     | 1     | $A - B$     |
| 1     | 0     | $A - B - 1$ |
| 1     | 1     | $A + 1$     |

Para realizar este ejercicio se deberán tener en cuenta las siguientes consideraciones:

- El diseño debe ser **modular**, con lo que habrá que diseñar primero una ALU básica para operandos de 1 bit y particularizarla, si es necesario, según la posición que ocupe dicho bit en los operandos  $A$  y  $B$ .
- Se podrán utilizar puertas lógicas, así como sumadores completos de 1 bit y multiplexores del tamaño que se necesite. Los multiplexores deberán tener señal **enable**.
- Siempre que se use un componente a modo de caja negra, será **obligatorio** indicar los nombres de cada una de sus entradas y salidas.
- Se dará prioridad a **reutilizar componentes** para que puedan realizar más de una operación.
- Se dará prioridad a utilizar el **menor número de componentes y del menor tamaño posible**.
- Habrá que calcular **razonadamente** el valor de las señales de control de los multiplexores utilizados (tabla de verdad y mapa de Karnaugh).

La **única** salida de la ALU es la siguiente:

- Resultado  $R = R_3R_2R_1R_0$ .
- En caso de producirse un resultado erróneo por desbordamiento, se ignorará este hecho y no se indicará de ninguna manera.

Se pide:

- Dibujar y explicar cómo sería una **miniALU para datos de 1 bit** que realice todas las operaciones posibles. Habrá que indicar y nombrar claramente cuáles son todas y cada una de las entradas y salidas de esta miniALU.
- Dibujar y explicar cómo habría que enlazar entre sí las 4 miniALUs para conseguir la **ALU completa de 4 bits**.

**Solución:**

a) Realizaremos primero una miniALU para datos de 1 bit, que después replicaremos en el siguiente apartado. Para la miniALU de peso  $i$ , las entradas serán  $A_i$ ,  $B_i$  y  $c_{in_i}$ . Las salidas serán  $R_i$  y  $c_{out_i}$ .

Lo primero que llama la atención es que todas las operaciones son aritméticas y se podrán hacer con un único sumador completo de 1 bit. Por tanto, en este caso, no será necesario utilizar un multiplexor de 4 entradas para seleccionar el resultado correcto, ya que en todo momento se calculará un único resultado.

Por otro lado, al sumador completo habrá que proporcionarle los operandos correctos. Si desarrollamos las operaciones para adaptarlas a los tres operandos del sumador, tenemos lo siguiente:

| $S_1$ | $S_0$ | Operación                                           | $a$   | $b$         | $c_{in0}$ |
|-------|-------|-----------------------------------------------------|-------|-------------|-----------|
| 0     | 0     | $A + B = A + B + 0$                                 | $A_i$ | $B_i$       | 0         |
| 0     | 1     | $A - B = A + \bar{B} + 1$                           | $A_i$ | $\bar{B}_i$ | 1         |
| 1     | 0     | $A - B - 1 = A + \bar{B} + 1 - 1 = A + \bar{B} + 0$ | $A_i$ | $\bar{B}_i$ | 0         |
| 1     | 1     | $A + 1 = A + 0 + 1$                                 | $A_i$ | 0           | 1         |

Puede verse que el operando  $A$  siempre aparece afirmado, con lo que podremos conectar directamente la entrada  $A_i$  de la miniALU a la entrada  $a$  del sumador.

Con respecto al operando  $B$ , a veces aparece afirmado, otras negado y otra anulado. Hay varias maneras de hacer esto:

- OPCIÓN 1: Podemos usar un multiplexor de 4 entradas, donde conectemos el valor  $B_i$  a la entrada 0, el valor  $\bar{B}_i$  a la entrada 1 y la conexión a tierra GND a la 2. La entrada 3 queda al aire. La señal de *enable* del multiplexor estaría conectada a VCC. En este caso, la señal de selección del multiplexor tiene 2 bits:

| $S_1$ | $S_0$ | Valor seleccionado | Señal de selección del mux |
|-------|-------|--------------------|----------------------------|
| 0     | 0     | $B_i$              | 00                         |
| 0     | 1     | $\bar{B}_i$        | 01                         |
| 1     | 0     | $\bar{B}_i$        | 01                         |
| 1     | 1     | 0                  | 10                         |

El mapa de Karnaugh del bit 1 de la señal de selección es:

| $S_1 \backslash S_0$ | 0 | 1 |
|----------------------|---|---|
| 0                    | 0 | 0 |
| 1                    | 0 | 1 |

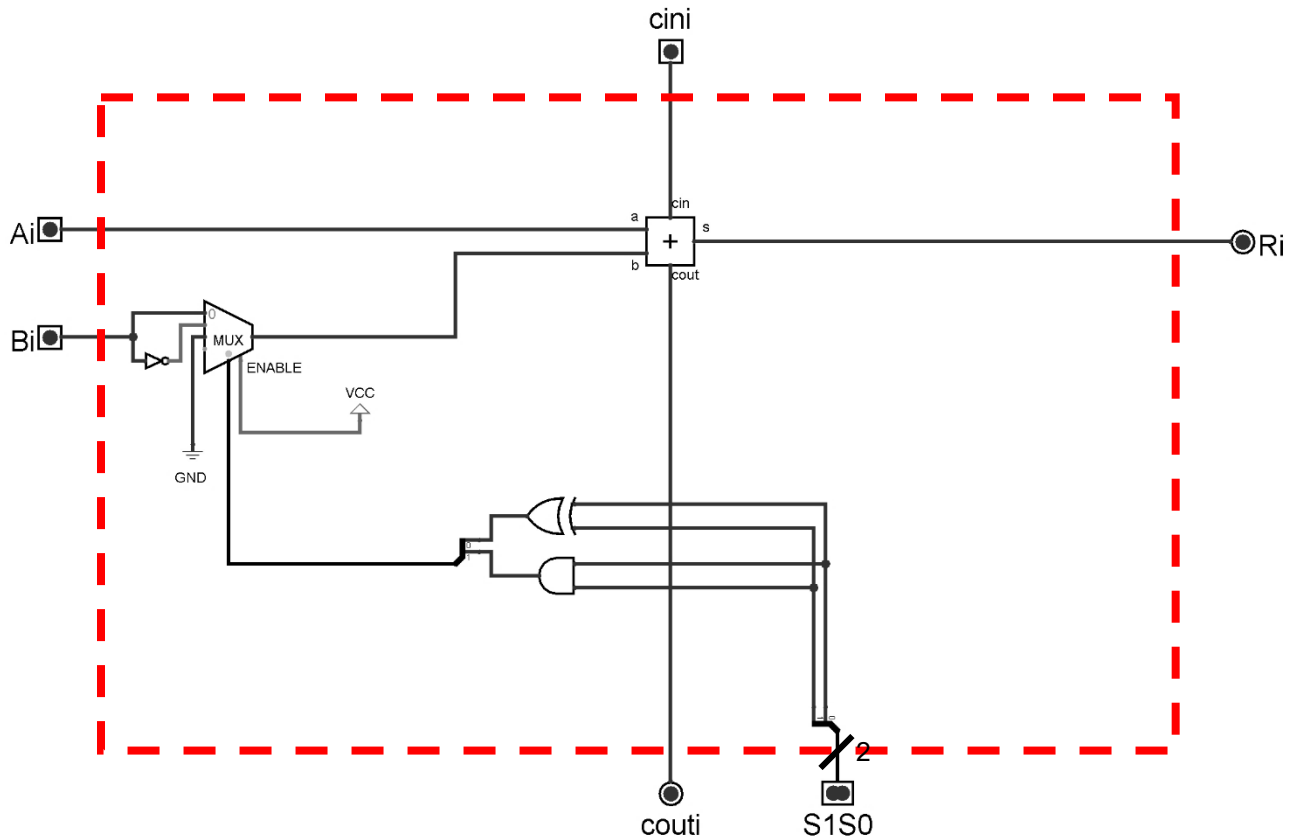
Este bit vale:  $S_1 S_0$ .

El mapa de Karnaugh del bit 0 de la señal de selección es:

| $S_1 \backslash S_0$ | 0 | 1 |
|----------------------|---|---|
| 0                    | 0 | 1 |
| 1                    | 1 | 0 |

Este bit vale:  $S_1\bar{S}_0 + \bar{S}_1S_0$ . También podemos darnos cuenta de que esta expresión se corresponde con la función lógica XOR, es decir,  $S_1 \oplus S_0$ .

En este caso, habría un multiplexor de 4 entradas, con 2 señales de control, más una puerta AND de 2 entradas y una puerta XOR de 2 entradas.



- OPCIÓN 2: Podemos usar un multiplexor de 2 entradas, donde conectemos el valor  $B_i$  a la entrada 0 y el valor  $\bar{B}_i$  a la entrada 1. A continuación, usaríamos un segundo multiplexor de 2 entradas, que eligiera entre el valor seleccionado por el primer multiplexor y la conexión a tierra. La señal de *enable* de ambos multiplexores estaría conectada a VCC.

En este caso, la señal de selección del primer multiplexor solo tiene 1 bit.

| $S_1$ | $S_0$ | Valor seleccionado | Señal de selección del primer mux |
|-------|-------|--------------------|-----------------------------------|
| 0     | 0     | $B_i$              | 0                                 |
| 0     | 1     | $\bar{B}_i$        | 1                                 |
| 1     | 0     | $\bar{B}_i$        | 1                                 |
| 1     | 1     | Da igual           | X                                 |

El mapa de Karnaugh de la señal de selección del primer multiplexor es:

|       |   | $S_0$ |   |
|-------|---|-------|---|
|       |   | 0     | 1 |
| $S_1$ | 0 | 0     | 1 |
|       | 1 | 1     | X |

Esta señal vale:  $S_1 + S_0$ .

En cuanto al segundo multiplexor, en la entrada 0 ponemos la salida del primer multiplexor y en la entrada 1 ponemos la conexión a tierra. Por tanto, la señal de selección de este multiplexor sería:

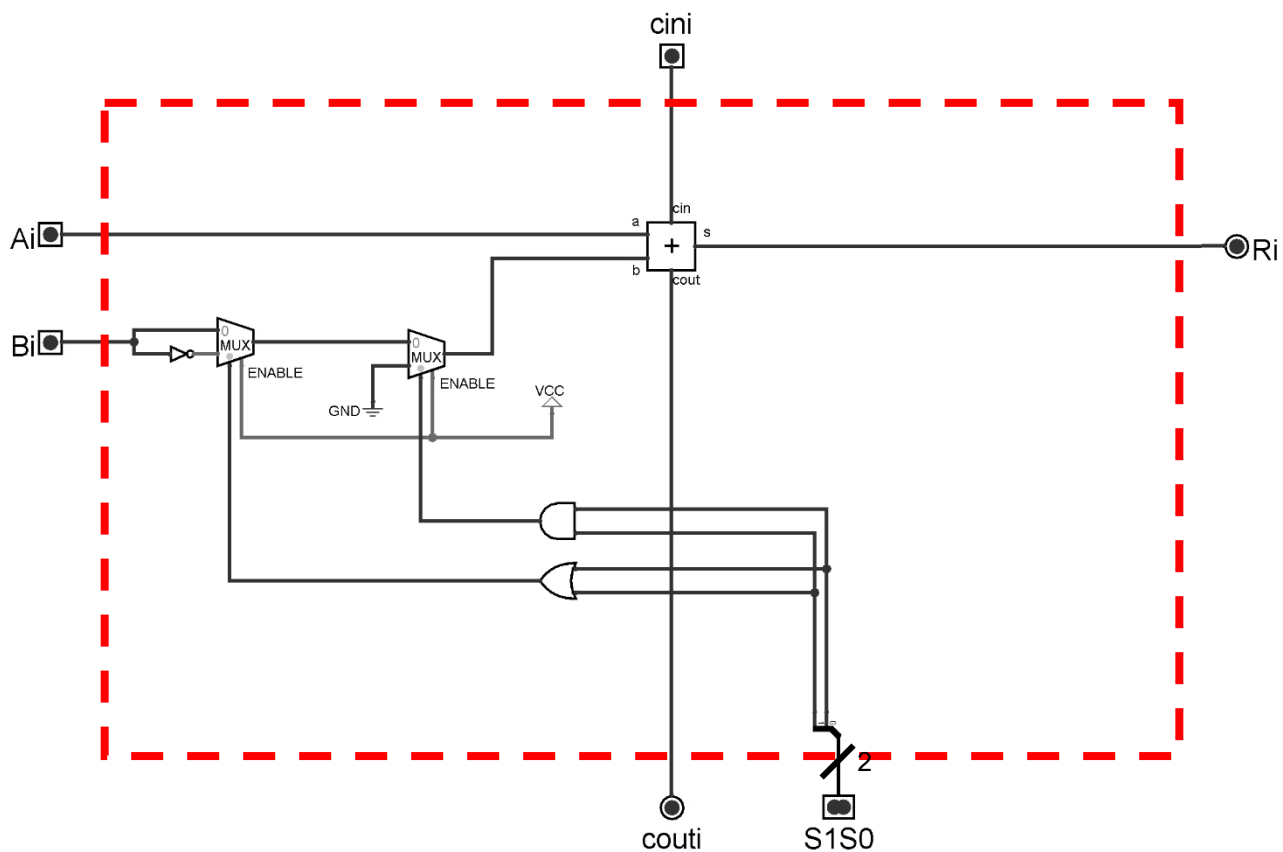
| $S_1$ | $S_0$ | Valor seleccionado                  | Señal de selección del segundo mux |
|-------|-------|-------------------------------------|------------------------------------|
| 0     | 0     | Valor seleccionado por mux anterior | 0                                  |
| 0     | 1     | Valor seleccionado por mux anterior | 0                                  |
| 1     | 0     | Valor seleccionado por mux anterior | 0                                  |
| 1     | 1     | 0                                   | 1                                  |

El mapa de Karnaugh de la señal de selección del segundo multiplexor es:

|       |   | $S_0$ |   |
|-------|---|-------|---|
|       |   | 0     | 1 |
| $S_1$ | 0 | 0     | 0 |
|       | 1 | 0     | 1 |

Esta señal vale:  $S_1 S_0$ .

Con esta opción, tendríamos dos multiplexores de 2 entradas, con 1 señal de selección cada uno, más una puerta OR de 2 entradas y una puerta AND de 2 entradas.



- OPCIÓN 3: Por último, podemos usar también un multiplexor de 2 entradas, como en el caso anterior, pero esa vez la señal de *enable* no sería constante, sino que se calcularía en función de  $S_1S_0$  para anularse en el caso 11. De esta forma, no sería necesario un segundo multiplexor.

| $S_1$ | $S_0$ | Valor seleccionado | Señal de selección del mux | <i>enable</i> |
|-------|-------|--------------------|----------------------------|---------------|
| 0     | 0     | $B_i$              | 0                          | 1             |
| 0     | 1     | $\bar{B}_i$        | 1                          | 1             |
| 1     | 0     | $\bar{B}_i$        | 1                          | 1             |
| 1     | 1     | Ninguno            | X                          | 0             |

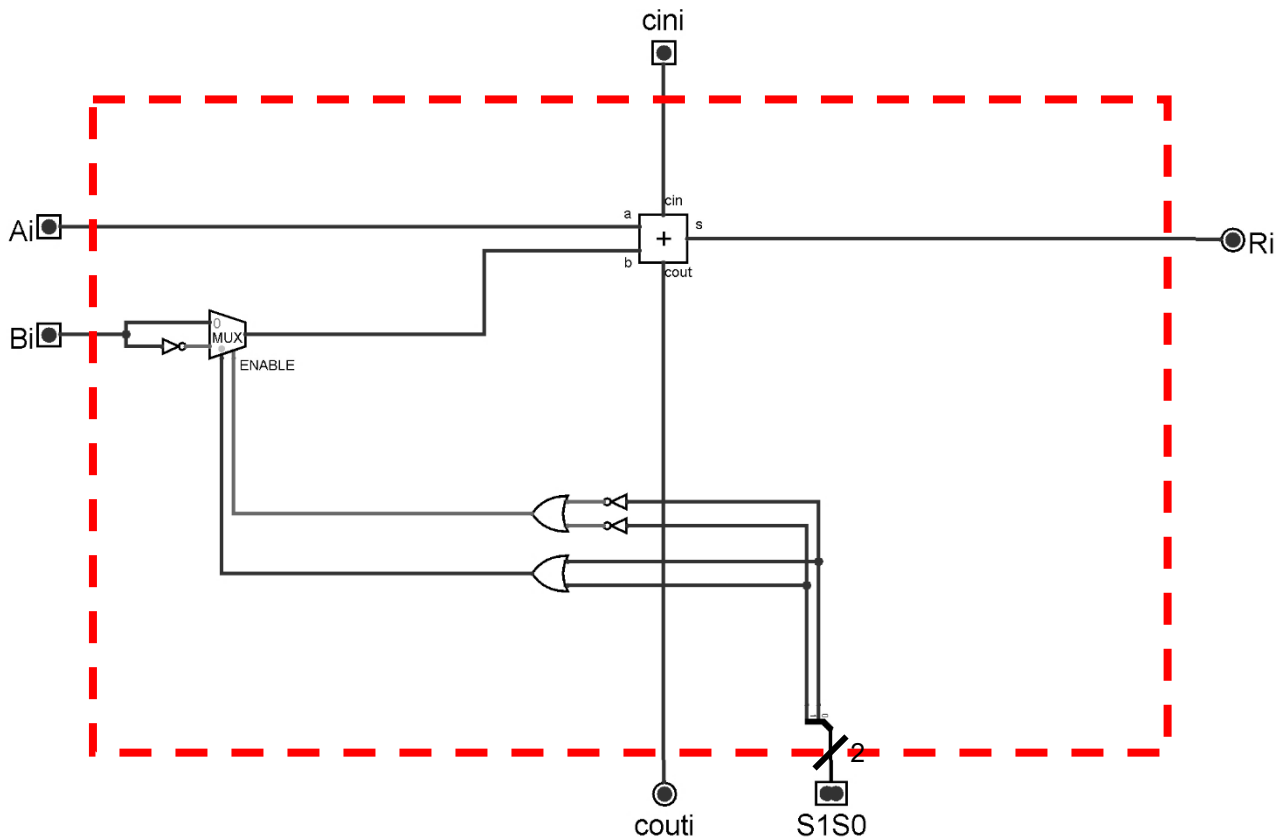
La señal de selección tiene un mapa de Karnaugh idéntico a otro resuelto previamente, con lo que ya sabemos que su valor es:  $S_1 + S_0$ .

En cuanto al *enable*:

| $S_1 \backslash S_0$ | 0 | 1 |
|----------------------|---|---|
| 0                    | 1 | 1 |
| 1                    | 1 | 0 |

Luego:  $enable = \bar{S}_1 + \bar{S}_0$ .

Por tanto, con esta tercera opción, habría un único multiplexor de 2 entradas, con 1 señal de selección, más 2 puertas OR de 2 entradas y 2 puertas NOT.



De las tres versiones, la tercera es la más reducida, pero todas se han valorado por igual en la corrección.

b) El acarreo de entrada de la miniALU de peso 0 a veces vale 0 y otras vale 1, con lo que tendremos que hallar el valor de  $c_{in_0}$  en función de  $S_1S_0$ .

| $S_1$ | $S_0$ | $c_{in_0}$ |
|-------|-------|------------|
| 0     | 0     | 0          |
| 0     | 1     | 1          |
| 1     | 0     | 0          |
| 1     | 1     | 1          |

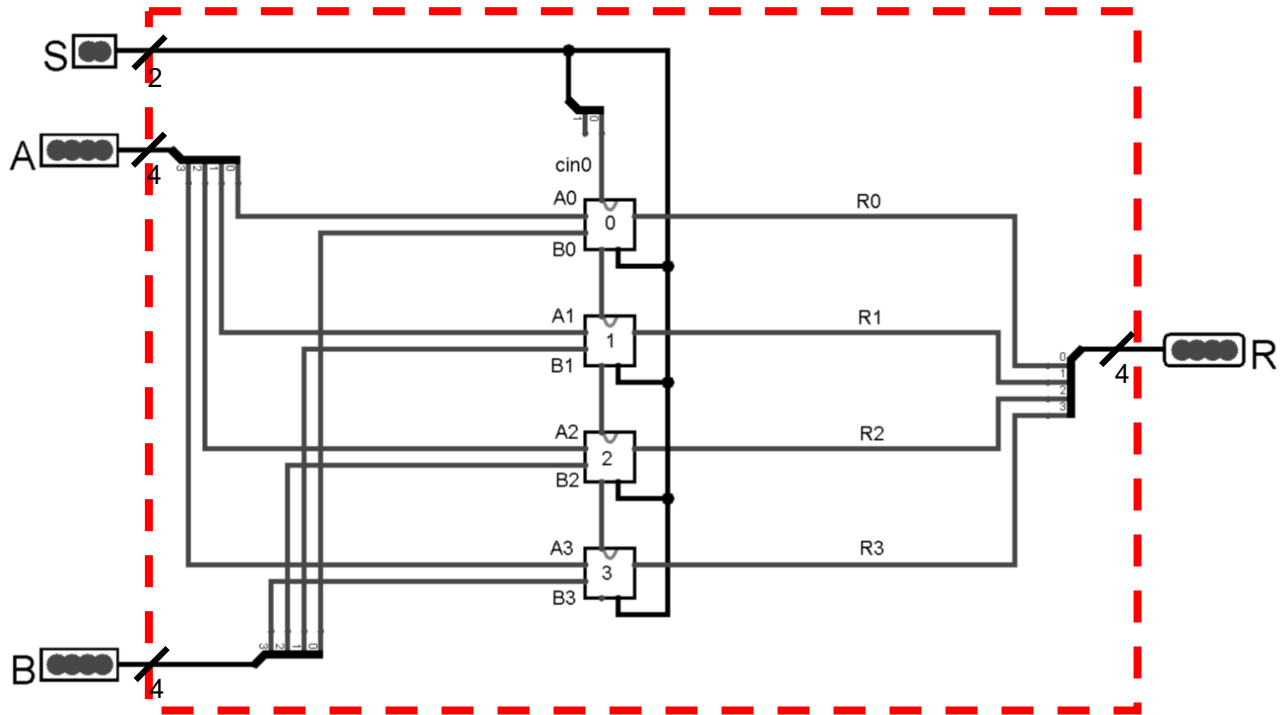
El mapa de Karnaugh correspondiente es:

|       |   | $S_0$ |   |
|-------|---|-------|---|
|       |   | 0     | 1 |
| $S_1$ | 0 | 0     | 1 |
|       | 1 | 0     | 1 |

Por tanto:

$$c_{in_0} = S_0$$

El circuito completo es:



Como es habitual en este tipo de ejercicios, se ha conectado el acarreo de salida de una miniALU con el de entrada de la miniALU siguiente. El acarreo de salida de la miniALU de peso 3 se queda al aire y no forma parte del resultado (no es una salida), ya que estamos trabajando en complemento a 2.